

Unidade I

1 QUALIDADE E PRODUTIVIDADE DE SOFTWARE

A **qualidade em software** refere-se ao grau em que um produto atende às **necessidades e expectativas dos usuários**, estando relacionada diretamente aos requisitos não funcionais (RNF) do software. O software de qualidade apresenta confiabilidade, usabilidade, eficiência, manutenibilidade, portabilidade e segurança.

A **produtividade em software** está relacionada à **eficiência do processo de desenvolvimento**, isto é, à **quantidade de produto** (funcionalidades, linhas de código, entregas) gerada em relação aos recursos utilizados (tempo, esforço, equipe e custo). Quanto **maior o valor que a equipe entrega em menos tempo e com menos recursos, maior é a produtividade**.

O equilíbrio ideal é buscar alta produtividade com qualidade sustentável, adotando boas práticas de engenharia de software, automação de testes e melhoria contínua de processos.

1.1 Demanda, qualidade e produtividade

A **qualidade de software** refere-se à **avaliação de um atributo próprio de um determinado componente de software**, de modo que ele atenda aos requisitos, seja livre de defeitos e entregue valor ao usuário final.

Um **produto de alta qualidade normalmente tem um custo elevado**, o que compromete sua competitividade no mercado; por outro lado, a redução da qualidade aumenta o número de defeitos e, conseqüentemente, o aumento do custo na logística reversa para manutenção, o que leva a um aumento no custo do produto.

Para atender à demanda de um determinado produto, a produção terá que acompanhar e, por vezes, reduzir o custo da produção para que o produto continue competitivo no mercado. Para aumentar a produção, é necessário aumentar o volume produzido. Para isso, deve-se melhorar a produtividade, o que implica em, por exemplo, produzir, no mesmo período ou conforme a necessidade, uma quantidade superior à do período anterior.

A produtividade de software mede a eficiência com que a equipe de desenvolvimento transforma requisitos funcionais (RF) em funcionalidades entregues.

Empresas enfrentam este **desafio: aumentar a produção em menor tempo a custos baixos. Para isso, abrem mão de alguns critérios de qualidade**, tais como suporte, inspeção, manutenção, testes, diagnósticos e outros.

1.2 Importância da qualidade no desenvolvimento ágil

A adoção de metodologias ágeis (como Scrum, XP, FDD, Kanban e outras), também chamadas de metodologias adaptativas, mudou o panorama do desenvolvimento de software, priorizando entregas contínuas e adaptação a mudanças.

Essa agilidade não deve ocorrer em detrimento da excelência técnica. Pelo contrário, a qualidade assume uma posição imperativa e intrínseca no desenvolvimento ágil.

No método ágil, a qualidade não é obtida na etapa final (como no modelo cascata), mas se constrói de forma contínua e integrada. Sem um rigoroso foco na garantia da qualidade do software (GQS) – do inglês, software quality assurance (SQA) –, a velocidade inicial rapidamente se transforma em débito técnico, metáfora para o custo futuro de refatoração e correção que se acumula quando o código de baixa qualidade é priorizado em detrimento do design.

1.2.1 Controle da qualidade: garantia de qualidade de software (GQS) ou software quality assurance (SQA)

A garantia da qualidade de software (GQS) é um conjunto sistemático de atividades planejadas e implementadas para garantir que os processos, produtos e artefatos de software estejam em conformidade com os padrões de qualidade estabelecidos pela organização e pelas normas internacionais.

SQA tem como objetivo assegurar que o software atenda aos requisitos especificados, minimizando defeitos e aumentando a confiabilidade e a satisfação do cliente.

Conceito de qualidade de software envolve a adequação do produto ao uso pretendido, sua conformidade com os requisitos do software e a capacidade do processo de desenvolvimento de produzir resultados consistentes e previsíveis.

Controle da qualidade, dentro do contexto da SQA, atua como um sistema de gestão que acompanha todas as fases do ciclo de vida do software



A qualidade desejada se refere à definição dos RNF do software, determinados para a sua construção. Os RNF correspondem às propriedades de qualidade e às restrições técnicas que influenciam o comportamento global de um sistema de software.

Principais RNF ISO 25000:

- **Funcionalidade:** é caracterizada por um conjunto coeso de operações relacionadas, desenvolvidas de acordo com a mesma base tecnológica e coerentes com a arquitetura do sistema.
- **Confiabilidade:** é a capacidade do software de manter seu desempenho e disponibilidade dentro de limites aceitáveis, mesmo sob condições imprevistas.
- **Usabilidade:** se refere à facilidade de uso, compreensão, atratividade, interação e aprendizado do software. A usabilidade é o principal critério a ser avaliado na interface humano-computador (IHC) e envolve aspectos de experiência do usuário (user experience – UX), acessibilidade e atratividade visual.
- **Segurança:** conjunto de práticas, medidas, mecanismos e políticas destinados à proteção do sistema de software contra ameaças, ataques e vulnerabilidades, internos e externos, contempla a confidencialidade, integridade, disponibilidade e autenticidade das informações.
- **Eficiência:** se refere ao desempenho do software e de suas funcionalidades, com relação a tempo de resposta, uso de recursos computacionais e capacidade de processamento.
- **Manutenibilidade:** corresponde à facilidade de implementar mudanças para atualizações do ambiente operacional, melhoria ou adaptação de funcionalidades a novos contextos.
- **Portabilidade:** refere-se a quanto

um determinado produto de software é portátil para outros ambientes operacionais, ou seja, o quanto pode ser adaptado ou executado em diferentes plataformas, ambientes operacionais ou dispositivos, com o mínimo de esforço técnico.

As eficazes práticas de desenvolvimento de software a seguir melhoram a relação entre qualidade e produtividade:

- Automação de testes: reduz o tempo de testes manuais e aumenta a cobertura de testes, melhorando a qualidade sem sacrificar a produtividade.
- Desenvolvimento ágil: promove entregas frequentes e iterativas, permitindo ajustes rápidos e contínuos que mantêm a qualidade alta e a produtividade constante.
- Reuso de código: São produzidos componentes reutilizáveis que podem acelerar o desenvolvimento e reduzir a probabilidade de erros.

Métodos na SQA:

- **Verificação e validação (V&V):** é uma das fases críticas do ciclo de desenvolvimento, porque **oferece garantias para que o software não apenas satisfaça as necessidades do usuário (validação), mas também esteja em conformidade com seus requisitos, especificações e padrões técnicos (verificação).**

Os testes de software são compostos por uma série de técnicas aplicadas durante todo o ciclo de desenvolvimento, que incluem:

- Teste unitário: tem foco nos componentes de código isolados.
- Teste de integração: verifica e testa a interface entre os módulos e componentes do sistema de software.
- Teste de sistema: se refere à validação dos RF e RNF do sistema de software completo.
- Teste de aceitação do usuário (user acceptance testing – UAT): assegura que o sistema atenda às necessidades do negócio e seja utilizável pelo cliente.

- **Análise estática e revisão de pares (peer reviews):** são atividades de SQA preventiva, projetadas para identificar e corrigir defeitos já no início do ciclo de desenvolvimento, quando o custo da correção é significativamente baixo.

- A análise estática se refere à inspeção do código, da documentação e dos modelos do sistema, para verificar se estão de acordo com os requisitos pré-definidos do software

— As revisões por pares são um processo sugerido pela SQA e por diversas metodologias ágeis, em que o trabalho do desenvolvedor, como código-fonte, documentação de requisitos ou modelos de arquitetura, é examinado por um ou mais colegas de trabalho.

— As revisões de código são revisões formais ou informais, auditorias, walkthroughs e inspeções para identificar problemas precocemente, incluindo inspeções (método estruturado baseado em papéis para detectar defeitos em artefatos, como código e documentação) e o uso de análise estática automatizada como complemento às inspeções manuais.

- **Gerenciamento de configuração de software (software configuration management – SCM):** o SCM é o processo de identificação, organização e controle sistemático de modificações nos artefatos do sistema durante o ciclo de vida. O SCM se baseia no controle de versão com ferramentas.

- **Gestão de artefatos e documentação:** artefato de trabalho que garante a rastreabilidade e a manutenibilidade do software. Inclui uma documentação técnica detalhada sobre a arquitetura de software, modelo de dados, especificações de requisitos e a documentação da interface de programação de aplicações (application programming interface – API).

- **Gestão de risco e plano de contingência:** essa atividade gerencial identifica, monitora, avalia e mitiga proativamente os riscos que ameaçam a qualidade e a agenda do projeto. — A mitigação de riscos consiste em definir regras e procedimentos de qualidade (como gateways de qualidade obrigatórios ou limites de cobertura de teste) para evitar a introdução de defeitos críticos. — Já o plano de contingência se refere à definição de ações de fallback (alternativa de segurança) e recursos a serem mobilizados quando um risco se materializa.

1.2.2 Custo da qualidade em engenharia de software e metodologias ágeis

Na engenharia de software, o custo da qualidade (cost of quality – CoQ) representa o conjunto de investimentos, esforços e recursos aplicados para garantir que o produto atenda aos requisitos do software, esse custo abrange todas as atividades voltadas à prevenção, avaliação e correção de falhas durante o ciclo de vida do software.

Nas metodologias ágeis, o custo da qualidade é entendido como parte do valor entregue ao cliente, ou seja, o equilíbrio entre o esforço investido em qualidade e o retorno percebido em confiabilidade, desempenho e satisfação do usuário final.

Scrum ou DevOps, a qualidade é continuamente avaliada por meio de iterações curtas, feedbacks constantes, testes automatizados e monitoramento contínuo.

Custo da qualidade pode ser entendido como o preço justo que o cliente esteja disposto a pagar. Esses custos podem ser divididos em:

- Custos de conformidade: são os custos necessários para assegurar que o software atenda aos padrões e requisitos de qualidade estabelecidos pela equipe e partes interessadas. Incluem: — Custos de prevenção: planejamento, revisões técnicas contínuas, projeto, definição de padrões de código, ferramentas de diagnóstico e testes, além de treinamento da equipe de desenvolvimento e dos usuários. Exemplo ágil: refinamento de backlog, sessões de pair programming e retrospectivas de sprint contribuem diretamente para a prevenção de defeitos. — Custos de avaliação: são os esforços para inspeções intra e interprocessos, produção, modelos de testes e diagnósticos, precisão, manutenção dos equipamentos e plano de mudanças no software. Exemplo ágil: revisões de incremento e testes de regressão automatizados em cada ciclo reduzem o risco de falhas não detectadas.
- Custos de não conformidade: representam os custos associados à falta de qualidade, como falhas, retrabalho ou defeitos que são identificados tardiamente. — Custos de falha interna: revisão de código, engenharia reversa, análise do modo como a falha ocorreu (root cause analysis, ou análise da causa-raiz), refatoração de código, atrasos na entrega e reparação de defeitos. Exemplo ágil: falhas identificadas em testes automatizados durante a integração contínua (pipeline de continuous integration – CI) exigem correção imediata antes da próxima integração. — Custos de falha externa: solução das queixas, devolução e substituição do produto, incidentes de segurança, manutenção da linha de suporte e serviços da garantia de qualidade e índices de satisfação. Exemplo ágil: após o report de um bug por parte de usuários, em fase de implantação, houve custos adicionais para a correção devido a análises de reversão, afetando, assim, o fluxo de valor.

1.3 Fatores, atributos e métricas da qualidade do software

Para determinar a qualidade, três conceitos distintos são utilizados: fator de qualidade, atributo de qualidade e métrica de qualidade.

1.3.1 Fator de qualidade

O fator de qualidade é o conjunto de uma ou mais características que influenciam a qualidade do produto de software. Um determinado fator de qualidade agrupa vários atributos específicos a serem implementados no componente de software.

Os fatores de qualidade do software podem ser divididos em dois grupos: • Fatores que podem ser medidos diretamente. Exemplo: defeitos por ponto de função. • Fatores que podem ser medidos indiretamente. Exemplo: usabilidade ou manutenibilidade.

1.3.2 Atributo da qualidade

O atributo da qualidade representa uma característica particular do produto, passível de ser medida e avaliada. Um ou vários atributos fornecem valores que **permitem avaliar um determinado fator de qualidade.**

– Sistema de software de gestão empresarial (ERP) um dos principais atributos da qualidade é a funcionalidade.

Algumas das seguintes métricas são consideradas no sistema de software de gestão empresarial:

- Usabilidade (RNF 1 – USAB): medição do índice de satisfação do usuário, de qualquer perfil, nas operações com o sistema de software; nível de implementação de requisitos que atendem às necessidades do usuário.
- Implementação (RNF 2 – IMPL): acompanhamento do cronograma, observando as metas referentes a prazos e custos.
- Integração com outras tecnologias (RNF 3 – INTI): medições de índice de integração da nova tecnologia com a já existente na empresa; grau de portabilidade para outros sistemas de software.
- Nível de atualizações (RNF 4 – NVAT): um baixo índice de atualizações indica robustez do projeto de software

1.3.3 Métricas da qualidade

Métricas são padrões de medidas dimensionadas para quantificar e qualificar o resultado de um determinado processo, produto ou serviço. A medida atende a um determinado padrão que expressa o resultado quantitativo de um atributo específico, seu formato e tipo de dados.

As dimensões das medidas estão relacionadas aos resultados das medições. Por exemplo: comprimento (metro – m); período (segundos – s); massa (grama – g); dígito (bit – b) e diversas outras. Elas podem ser combinadas e associadas para formar um resultado específico. Por exemplo: velocidade em m/s; armazenamento em bytes (1 byte = 8 bits) e taxa de transferência em bps (ou bit/s). Os dados coletados são analisados por engenheiros de software, comparados com médias anteriores e avaliados para verificar se ocorreu algum desvio no processo de software.

Quadro 2 – Principais métricas de qualidade de software

Métrica	Definição
Acurácia	Capacidade do produto de software de prover, com o grau de precisão necessário, resultados ou efeitos corretos ou conforme acordados
Adaptabilidade	Capacidade do produto de software de ser adaptado para diferentes ambientes especificados, sem necessidade de aplicação de outras ações ou meios além daqueles fornecidos para essa finalidade pelo software considerado
Adequação	Capacidade do produto de software de prover um conjunto apropriado de funções para tarefas e objetivos do usuário especificados
Analisabilidade	Capacidade do produto de software de permitir o diagnóstico de deficiências ou causas de falhas e a identificação de partes a serem modificadas
Apreensibilidade	Capacidade do produto de software de possibilitar ao usuário o aprendizado de sua aplicação
Atratividade	Capacidade do produto de software de ser atraente ao usuário
Auditabilidade	Facilidade no atendimento às normas que pode ser verificado
Autodocumentação	Nível em que o código-fonte fornece documentação significativa
Capacidade	Capacidade do produto de software de ser instalado em ambiente especificado e atender aos requisitos de funcionamento
Coexistência	Capacidade do produto de software de coexistir com outros produtos de software independentes, em um ambiente comum, compartilhando recursos comuns
Confiabilidade	A efetiva realização das funções de um programa em um nível de desempenho e precisão especificadas
Conformidade	Capacidade do produto de software de estar de acordo com normas, convenções ou regulamentações previstas em leis e prescrições similares relacionadas ao atributo do software
Eficácia	Capacidade do produto de software de permitir que usuários atinjam metas especificadas com acurácia e completitude, em um contexto de uso especificado
Eficiência	Quantidade de recursos e códigos de computação para realização da função. Quanto menor a quantidade, maior será o desempenho do software
Estabilidade	Capacidade do produto de software de evitar efeitos inesperados decorrentes de modificações no software
Expansibilidade	Nível em que o projeto arquitetural de dados ou procedimental pode ser estendido
Flexibilidade	Esforço necessário para modificar um programa operacional
Funcionalidade	Capacidade do produto de software de prover funções que atendam às necessidades explícitas e implícitas, quando o software estiver sendo utilizado sob condições especificadas
Generalidade	Âmbito da aplicação potencial dos componentes do programa
Integridade	Diz respeito ao acesso do software ou de dados por pessoas não autorizadas que pode ser controlado
Interoperabilidade	Capacidade do produto de software de interagir com um ou mais sistemas especificados

Métrica	Definição
Mantenabilidade	Esforço necessário para localizar e corrigir erros num programa
Manutenabilidade	Capacidade do produto de software de ser modificado. As modificações podem incluir correções, melhorias ou adaptações do software devido a mudanças no ambiente e nos seus requisitos ou especificações funcionais
Maturidade	Capacidade do produto de software de evitar falhas decorrentes de defeitos no software
Modificabilidade	Capacidade do produto de software de permitir que uma modificação especificada seja implementada
Modularidade	Diz respeito à independência funcional dos componentes do programa
Precisão	Nível de precisão dos cálculos e do controle que pode ser verificado
Operacionalidade	Capacidade do produto de software de possibilitar ao usuário operá-lo e controlá-lo
Produtividade	Capacidade do produto de software de permitir que seus usuários empreguem quantidade apropriada de recursos em relação à eficácia obtida, em um contexto de uso especificado
Portabilidade	Capacidade do produto de software de ser transferido de um ambiente operacional para outro
Rastreabilidade	Diz respeito à habilidade de rastrear uma representação de projeto ou componente real do programa
Recuperabilidade	Capacidade do produto de software de restabelecer seu nível de desempenho especificado e recuperar os dados diretamente afetados no caso de uma falha
Reutilização	Quanto de um programa ou parte dele pode ser reutilizado em outras aplicações
Satisfação	Capacidade do produto de software de satisfazer usuários, em um contexto de uso especificado Nota: satisfação é a resposta do usuário à interação com o produto e inclui atitudes relacionadas ao uso do produto
Segurança	Capacidade do produto de software de apresentar níveis aceitáveis de riscos de danos às pessoas, aos negócios, da propriedade, do ambiente, de proteger informações e dados, de modo que pessoas ou sistemas não autorizados não possam lê-los nem modificá-los e que não seja negado o acesso às pessoas ou sistemas autorizados
Testabilidade	É necessário testar um programa para sua validação, quando criado ou modificado, a fim de garantir que realize a função esperada
Usabilidade	Capacidade do produto de software de ser compreendido, aprendido, operado e atrativo ao usuário, quando usado sob condições especificadas
Utilização	Esforço necessário para aprender, operar, preparar entradas e interpretar saídas de um programa

Exemplo de aplicação

Para **determinar as métricas da qualidade** são considerados os atributos da **qualidade do requisito funcional** gerenciamento do BD: **implementação (IMPL)**, **integração com outras tecnologias (INTI)** e **nível de atualizações (NVAT)**.

- Implementação (IMPL): medida direta de produção, em períodos de dias (d.), comparada com as metas definidas no cronograma.
- Integração com outras tecnologias (INTI): podem ser efetuadas medidas indiretas, tais como medidas de portabilidade do BD, que definem na prática os sistemas operacionais e outros gerenciadores de BDs em que o software irá operar.
- Nível de atualizações (NVAT): são medidas diretas de versionamento do software que indicam o índice de mudanças no código-fonte. São definidas as versões que acompanham o desenvolvimento do software e as versões liberadas ao cliente. O número de mudanças em um determinado período pode indicar instabilidade do software.

2 ENGENHARIA DE SEGURANÇA DE SOFTWARE

A engenharia de segurança de software planeja, desenvolve e mantém sistemas seguros e confiáveis, protegendo dados e funcionalidades contra falhas, vulnerabilidades e ataques. Tem como objetivo garantir a integridade, confidencialidade, autenticidade e disponibilidade das informações ao longo de todo o ciclo de vida do software.

As técnicas aplicadas fornecem práticas como análise de ameaças, modelagem de riscos, testes de penetração e criptografia

2.1 Gerenciamento e segurança: análise do risco, plano de contingência e seguro

Gerenciamento de risco (GR), o plano de contingência (PC) e a seguridade (seguro) aplicados ao software envolvem práticas sistemáticas que visam identificar, avaliar e mitigar riscos que possam comprometer a qualidade, a continuidade e a confiabilidade dos sistemas computacionais, para a sustentabilidade operacional e a proteção dos ativos de informação.

O risco é conceituado como “um evento ou condição incerta que, se ocorrer, tem um efeito positivo ou negativo sobre ao menos um dos objetivos a ser cumprido”, ou seja, a probabilidade de perigo, uma ameaça física para o homem ou para o meio em que se encontra, ou a probabilidade de insucesso em função de um acontecimento ou incidente que acarrete indenização.

A análise do risco consiste na identificação, categorização e avaliação das ameaças e vulnerabilidades que podem afetar o software, considerando tanto aspectos técnicos (falhas de código, ataques cibernéticos, indisponibilidade de servidores) quanto organizacionais (erros humanos, falhas de comunicação, requisitos mal definidos).

Elementos	Definições
Ameaça	Qualquer evento, pessoa ou condição que possa causar um resultado indesejado ou comprometer um artefato ou o processo de software. Exemplo: atraso no fornecimento de uma API de fornecedor terceirizado
Probabilidade	É a chance de uma ameaça se concretizar durante o desenvolvimento de uma funcionalidade, sprint ou o comprometimento com o ciclo de entregas. Exemplo: há grande probabilidade de que a implementação de um novo código contenha regressões, se testes unitários não forem concluídos
Impacto	É a medida do dano ou custo (em tempo, dinheiro, reputação ou funcionalidade) que a ocorrência do risco pode acarretar ao produto de software, ao desempenho da equipe e/ou ao compromisso de entregas. Exemplo: impacto da mudança na ocorrência de uma falha de software que impede a implantação do software
Incerteza	O grau de incerteza é medido de acordo com a falta de informações claras sobre um determinado requisito, uma dependência técnica ou a viabilidade de uma solução. Nas metodologias ágeis, isso é frequente e conhecido como spike (em inglês: atividade de pesquisa ou experimentação técnica), com objetivo de converter a incerteza em informação
Ação alternativa	É a estratégia de mitigação ou plano de contingência (rollback, que significa ação corretiva) definido para neutralizar ou reduzir o risco. Exemplo: implementar um recurso de fallback (recuo) para serviços de terceiros, ou refatorar o código para reduzir o débito técnico

Plano de contingência, um documento estratégico que define ações preventivas e corretivas para garantir a continuidade do serviço e a recuperação dos sistemas após incidentes. Inclui rotinas de backup e recuperação de dados, estratégias de redundância de servidores, planos de resposta a incidentes de segurança e mecanismos de failover (processo automático de transferência de operações e dados entre servidores), para minimizar o tempo de inatividade e garantir a continuidade dos serviços. Trata-se de práticas de DevOps e de integração contínua/entrega contínua, que facilitam a rápida correção de falhas, com objetivo de reduzir a exposição a riscos.

O **seguro** aplicado a software (ou seguro cibernético) surge como uma ferramenta financeira de mitigação de risco, destinada a cobrir prejuízos decorrentes de incidentes de segurança, como vazamento de dados, interrupção de serviços, ataques de ransomware e falhas de infraestrutura.

2.2 Gerenciamento de riscos do projeto

O risco pode ser conceituado como a combinação da probabilidade da ocorrência de um evento e de suas consequências sobre os objetivos do projeto. Em termos quantitativos, o risco pode ser representado pela relação:

Risco = Probabilidade (P) × Impacto (I)

- Probabilidade (P): o quanto é provável que o risco ocorra (por exemplo: baixa, média ou alta).
- Impacto (I): quão severas serão as consequências, caso o risco ocorra (por exemplo: pequeno, moderado ou crítico).

Nos modelos prescritivos (cascata, espiral e outros), o gerenciamento do risco é aplicado em fases específicas do ciclo de desenvolvimento, enquanto nas metodologias ágeis (Scrum, XP, Kanban, FDD e outros), o monitoramento e a mitigação de riscos ocorrem de forma iterativa, ou seja, promovem ajustes rápidos ao longo do ciclo de desenvolvimento da funcionalidade ou da sprint.

Principais processos de gerenciamento dos riscos do projeto:

- 1)Planejar o gerenciamento do risco: definir como as atividades de risco serão conduzidas no projeto, considerando também a dinâmica das metodologias ágeis.
- 2)Identificar os riscos: levantar riscos técnicos, de negócio e de processo por meio de reuniões de planejamento, histórias de usuário (user story) e retrospectivas.
- 3)Realizar análise qualitativa e quantitativa dos riscos: priorizar riscos (baixo, médio e alto), conforme seu impacto (pequeno, moderado ou crítico) nas mudanças e/ou nas entregas de curto prazo.
- 4)Planejar as respostas aos riscos: definir estratégias como spikes técnicos, automação de testes e pair programming para mitigar incertezas.
- 5)Implementar respostas aos riscos: aplicar as ações planejadas de mitigação e contingência durante as iterações. Em um ambiente ágil, isso ocorre durante as sprints, com a equipe ajustando atividades, prioridades e recursos.

2.3 Governança em TI

A governança em tecnologia da informação (TI) se baseia em um conjunto de estratégias, políticas para uso da tecnologia, estruturas e processos decisórios com o intuito de apoiar e expandir os objetivos estratégicos e operacionais da

organização. Frameworks como COBIT 2019 e ITIL v4 desempenham papéis fundamentais, fornecendo diretrizes para a governança e a gestão de serviços de TI. COBIT 2019 possui fatores de design que permitem a personalização do sistema de governança de TI. O Control Objectives for Information and Related Technologies (COBIT) é um framework para gestão e controle de ambientes de TI empresariais, alinhando as estratégias de negócios com as estratégias de TI. ITIL v4, por sua vez, enfatiza a gestão de serviços de TI com foco na entrega de valor e na melhoria contínua. O Information Technology Infrastructure Library (ITIL) integra formas de trabalho como lean, metodologias ágeis e DevOps.

2.4 Estratégia para suporte de software

Ela deve englobar a gestão proativa de riscos, a otimização da experiência do usuário (UX) e a manutenção da satisfação do cliente a longo prazo.

Quadro 4 – Princípios de integração da engenharia de segurança e metodologias ágeis

Princípio	Objetivo	Prática
Segurança tratada como RNF	A segurança é tratada como parte da qualidade do software	Definir histórias de usuário com critérios de segurança (security stories)
Resiliência e continuidade	O sistema deve suportar falhas (bugs) e ataques sem perda crítica	Implementar mecanismos de rollback (reverter), redundância de código e dados e logs de auditoria
Iteratividade e resposta rápida	Entregas curtas favorecem correções e reforços de segurança contínuos	Aplicar DevSecOps e pipelines de CI/CD com testes de segurança automatizados
Cultura de responsabilidade compartilhada	A segurança é de toda a equipe de desenvolvimento, incluindo cliente e usuários, não apenas especialista	Pair programming e revisões de código com foco em vulnerabilidades

2.5 Retorno de investimento (ROI) no ciclo de entrega ágil

O retorno de investimento (ROI) é observado diretamente na eficácia da aplicação e na sua capacidade de suportar os objetivos estratégicos da organização.

ROI é maximizado pela adoção de práticas ágeis que garantem que o capital investido se traduza rapidamente em resultados mensuráveis como:

- Minimizar custo de refatoração e feedbacks rápidos: consiste na entrega de sprints em ciclos curtos, permitindo que stakeholders validem as funcionalidades de forma rápida, reduzindo, assim, o risco de desenvolver um recurso errado, economizando tempo e recursos.

- Aumento da produtividade e eficiência operacional: um software que atende com precisão e clareza aos requisitos de negócio se traduz diretamente em maior velocidade de adoção, redução de erros operacionais

2.6 Métodos, ferramentas e técnicas (MF&T): melhoria da qualidade de software

Plano de garantia da qualidade (GQS), contemplando: • padrões de codificação baseados em Cleanroom; • revisões de código (peer review) obrigatórias via GitHub; • checklist em conformidade com ISO/IEC 25010 (funcionalidade, confiabilidade, desempenho e segurança); • auditorias internas de qualidade a cada três sprints.

Métricas de qualidade (ferramentas aplicáveis)

No quadro a seguir são apresentadas algumas das ferramentas aplicáveis para mensurar a evolução da qualidade:

Quadro 5 – Métricas da qualidade e ferramentas aplicáveis

Métrica	Ferramenta/desenvolvedor	Características
Eficiência	Jira/Atlassian	Mede o tempo entre a criação de uma tarefa e sua conclusão Eficiência na remoção de defeitos antes da entrega; permite o acompanhamento de bugs, tarefas e qualidade do processo
Eficiência	GitHub – modo Kanban/ GitHub, Inc. (Microsoft)	Mede o throughput (taxa de conclusão) Tempo médio para corrigir falhas Acompanhamento visual do fluxo de trabalho
Desempenho	GanttProject/ Bartłomiej Nawrocki	Geração de cronogramas, distribuição de responsabilidades e tarefas (ou sprints) Checklist; medição de velocidade da sprint (story points entregues)



Quadro 6 – Metas a serem atingidas

Categoria	Meta da qualidade	Métrica/medida	Tipo de custo
Eficiência do processo	Reduzir em 20% o tempo médio de resolução de tarefas em três sprints	Lead time	Prevenção e avaliação
Remoção de defeitos antes da entrega	Aumentar em 30% a taxa de correção de defeitos detectados internamente, antes da entrega ao cliente	Taxa de defeitos removidos antes da entrega	Falhas internas
Produtividade da equipe	Elevar a velocidade média da sprint em 10%, mantendo a qualidade estável	Story points entregues/sprint	Avaliação
Confiabilidade pós-entrega	Diminuir em 25% as falhas externas reportadas nos dois primeiros meses após a entrega	Quantidade de bugs reportados pós-entrega	Falhas externas

Unidade II

3 NORMAS E PADRÕES DA QUALIDADE

A qualidade de software não é um conceito abstrato, é o resultado da aplicação de métodos e métricas fundamentadas em normas e padrões bem definidos.

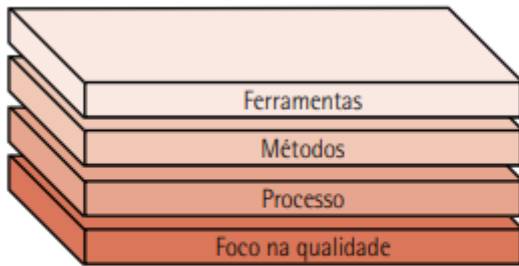


Figura 4 – Camadas da engenharia de software

A espinha dorsal da garantia da qualidade explora o universo das normas e padrões internacionais que regem o desenvolvimento de software. Modelos organizacionais como a International Organization for Standardization (ISO) e o Institute of Electrical and Electronics Engineers (IEEE), além de modelos como o Capability Maturity Model Integration (CMMI®) estabelecem uma linguagem comum, baseada nas melhores práticas e requisitos essenciais para todos os estágios do ciclo de vida do software.

3.1 Processos do ciclo de vida do software: NBR ISO/IEC 12207

É uma referência direta ao título oficial da norma brasileira (NBR), que adota o padrão internacional ISO/IEC 12207.

A Norma NBR ISO/IEC 12207 – Processos do ciclo de vida do software, publicada em 1995, foi elaborada com o objetivo de fornecer uma estrutura padronizada e uma linguagem comum para todos os participantes envolvidos no desenvolvimento e na manutenção de software. Essa linguagem comum é estabelecida por meio de processos claramente definidos, que servem como referência para a organização e o controle das atividades ao longo do ciclo de vida do software. O framework da

norma ISO/IEC 12207 fornece diretrizes abrangentes para os processos envolvidos no desenvolvimento, manutenção e operação de sistemas de software

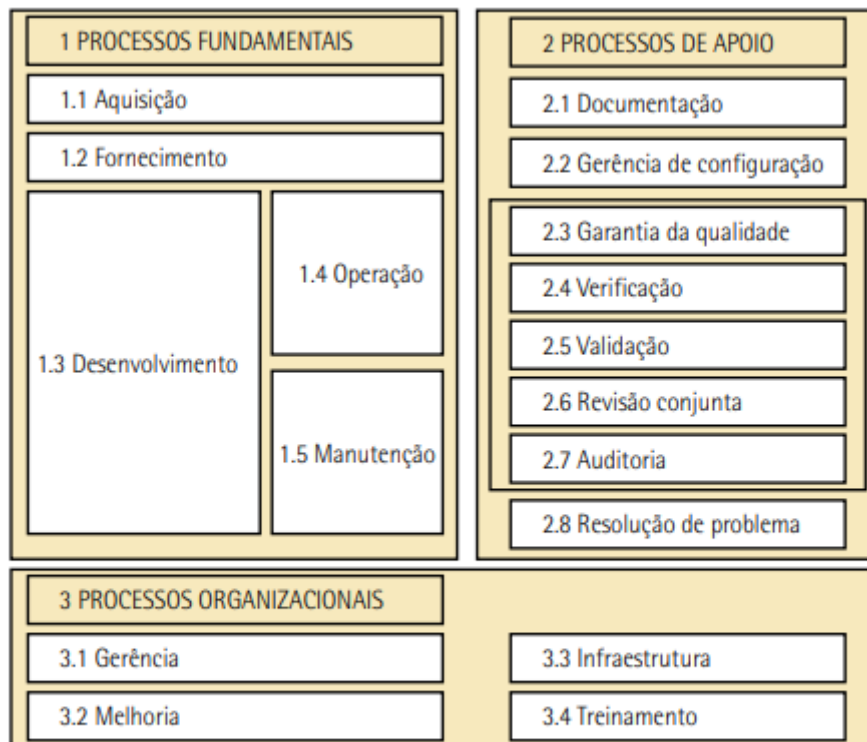


Figura 5 – NBR ISO/IEC 12207 – Processos do ciclo de vida do software

Estrutura de processos da ISO/IEC 12207

1)Processos fundamentais: Envolvem as atividades essenciais que transformam requisitos em soluções tecnológicas.

1.1)Aquisição: trata da obtenção de software, produtos ou componentes de terceiros, incluindo especificação de requisitos, seleção de fornecedores, negociação e aceitação dos produtos adquiridos.

1.2)Fornecimento: refere-se ao fornecimento de software a clientes ou parceiros, contemplando a elaboração de propostas, o planejamento do contrato e a entrega do produto final.

1.3)Desenvolvimento: abrange o conjunto de atividades destinadas à criação do software, desde a definição de requisitos, análise, projeto, codificação, integração e testes, até a entrega do sistema.

1.4)Operação: engloba a execução e o monitoramento contínuo do sistema em ambiente de produção, garantindo sua disponibilidade, desempenho e suporte ao usuário.

1.5)Manutenção: inclui as atividades de correção, adaptação, evolução e prevenção de falhas, assegurando que o software permaneça funcional e alinhado às mudanças tecnológicas e organizacionais.

2)Processos de apoio: os processos de apoio são aqueles que fornecem suporte aos processos fundamentais, assegurando a qualidade, rastreabilidade e conformidade do software com normas e padrões estabelecidos.

2.1)Documentação: compreende a criação, os modelos, a atualização e o controle de documentos técnicos e gerenciais que descrevem o software, seus requisitos, arquitetura e operação.

2.2)Gerência de configuração: controla versões, releases e mudanças que ocorrem no ciclo de vida do software, garantindo integridade e rastreabilidade ao longo do ciclo.

2.3)Garantia da qualidade: estabelece políticas, práticas e auditorias que asseguram a conformidade do produto e dos processos com os padrões de qualidade definidos.

2.4)Verificação: avalia se as saídas de uma fase atendem às especificações definidas na fase anterior, garantindo coerência entre requisitos e implementações.

2.5)Validação: confirma se o produto final atende às necessidades e expectativas do cliente, consolidando o processo de verificação e validação (V&V).

2.6)Revisão conjunta: é uma revisão formal e estruturada por parte dos stakeholders para avaliar o andamento do projeto, detectar inconsistências e confirmar o alinhamento com os requisitos.

2.7)Auditoria: realiza uma análise independente e sistemática dos processos e produtos de software.

2.8)Resolução de problemas: trata da identificação e correção de defeitos, falhas, erros e não conformidades, promovendo a estabilidade e confiabilidade do software.

3)Processos organizacionais: os processos organizacionais são de caráter estratégico e abrangem o nível mais alto da gestão de software

3.1)Gerência: abrange o planejamento, controle e monitoramento de recursos humanos, financeiros, tecnológicos e materiais

3.2)Melhoria: foca na avaliação e aperfeiçoamento contínuo dos processos de software, buscando maior eficiência, qualidade e maturidade organizacional.

3.3)Infraestrutura: compreende os elementos essenciais e prioritários para o desenvolvimento e a operação de sistemas de software, definidos pela gerência e alinhados às necessidades organizacionais.

3.4)Treinamento: garante que os profissionais envolvidos possuam as competências, as habilidades e os conhecimentos atualizados necessários à execução das atividades com qualidade e segurança.

3.2 Requisitos ergonômicos para trabalhos com computadores: NBR ISO/IEC 9241-11

A ergonomia cognitiva é uma área da ergonomia que se concentra no estudo e na melhoria da interação entre os seres humanos e os sistemas cognitivos, como a mente, a memória, a atenção e a percepção. Um aspecto importante da ergonomia cognitiva em sistemas computacionais reside no design de interfaces de usuário de alta usabilidade. O desenvolvimento de interfaces otimizadas é um imperativo técnico, pois permite a redução da carga cognitiva do usuário.

NBR ISO/IEC 9241-11 define a usabilidade a partir de três pilares mensuráveis:

- eficácia: a precisão e a completude com que os usuários alcançam objetivos específicos;
- eficiência: os recursos (tempo, esforço mental) gastos para atingir esses objetivos;
- satisfação: o conforto e a aceitação do sistema pelos usuários.

Estabelece uma metodologia baseada em contexto de uso que permite aos desenvolvedores e avaliadores:

- identificar os requisitos ergonômicos no início do ciclo de vida do desenvolvimento;
- avaliar objetivamente a usabilidade de produtos de software e sistemas;
- garantir que os produtos não apenas funcionem corretamente, mas também promovam produtividade, saúde e aceitação.

No domínio do design centrado no usuário (DCU), uma das abordagens teóricas mais influentes é a engenharia cognitiva.

Processo de desenvolvimento criando seu próprio modelo mental do sistema, denominado modelo de design. Esse modelo, mostrado na figura 7, é construído com base em uma análise prévia dos modelos de usuário (o entendimento das capacidades e expectativas do usuário) e dos modelos de tarefa (a representação das atividades a serem executadas). Por fim, no modelo da imagem do sistema a ser implementado, a representação física do sistema na interface é estruturada a partir da materialização do modelo de design, visando alinhar a percepção do usuário com a funcionalidade pretendida pelo sistema. O usuário então interage com essa imagem do sistema e cria seu modelo mental da aplicação, chamado de modelo do usuário.

A figura a seguir mostra o **processo de design** na abordagem da engenharia cognitiva:

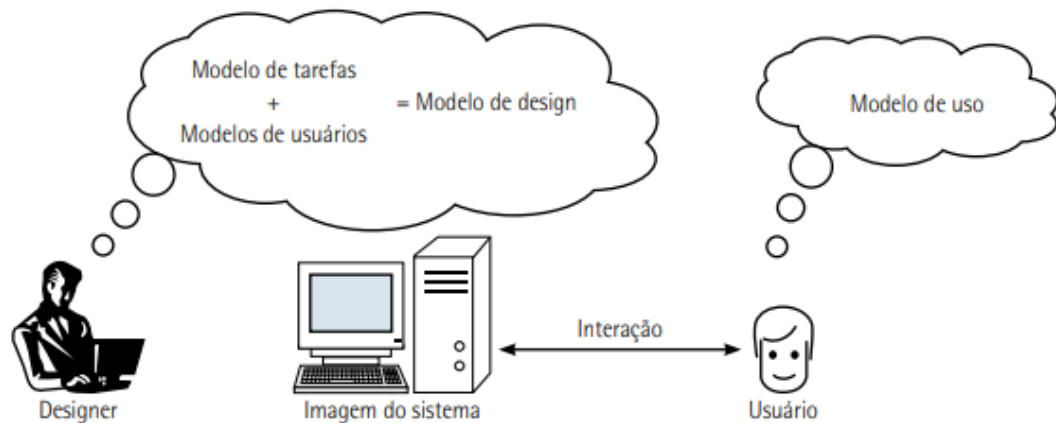


Figura 7 – Processo de design: modelo de interação da engenharia cognitiva

3.3 Avaliação do produto de software: ISO 9126, ISO 14598 e ISO 25000

A especificação e a avaliação da qualidade do produto de software constituem elementos que asseguram que o sistema atenda aos níveis de desempenho e confiabilidade esperados. Cada característica relevante deve ser claramente especificada, mensurada e avaliada utilizando, sempre que possível, métricas validadas ou amplamente aceitas, conforme estabelecido na NBR ISO/IEC 9126-1 e nas normas complementares ISO/IEC 14598 e ISO/IEC 25000, que evoluíram para consolidar um modelo integrado de avaliação da qualidade de produtos de software. Com o objetivo de padronizar esses processos (avaliação do produto de software), foram estabelecidas as seguintes normas: em 1991, a NBR ISO/IEC 9126 (Qualidade do produto de software), e em 1998, a NBR ISO/IEC 14598 (Avaliação de produto de software).

NBR ISO/IEC 9126 consolidou o modelo de avaliação da qualidade de software por meio de métricas internas, externas e de qualidade em uso.

Essa estrutura demonstra como cada parte do processo está associada a um conjunto específico de normas, que orientam desde o apoio à avaliação até a mensuração da qualidade em uso.

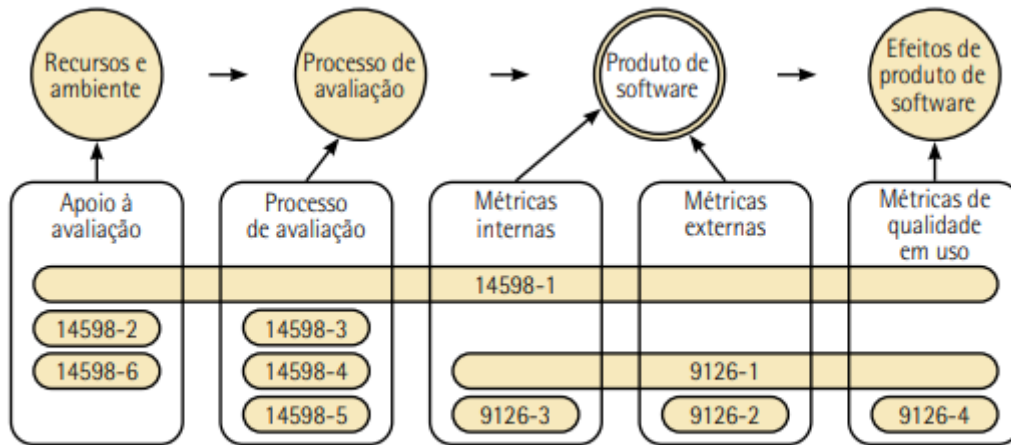


Figura 8 – Relação entre as NBR ISO/IEC 9126 e NBR ISO/IEC 14598

3.3.1 NBR ISO/IEC 9126 – Qualidade do produto da engenharia de software

As características de qualidade do produto de software, conforme definidas pela NBR ISO/IEC 9126, constituem um referencial para a especificação de RNF e RF em ambientes de desenvolvimento orientados por processos tradicionais e metodologias ágeis.

O modelo de qualidade proposto pela norma decompõe a qualidade do produto de software em um conjunto de características e subcaracterísticas internas, externas e de uso, que podem ser incorporadas de forma incremental nas iterações do ciclo de desenvolvimento ágil.

A avaliação da qualidade deve empregar métricas e medições adequadas aos objetivos estratégicos do projeto, ao contexto tecnológico adotado e aos cenários reais de uso do software.

NBR ISO/IEC 9126 no contexto ágil:

- Avaliação contínua da qualidade: nas metodologias ágeis, a qualidade é verificada de forma iterativa ao longo dos sprints. A NBR ISO/IEC 9126 oferece um modelo estruturado de avaliação com características e subcaracterísticas como funcionalidade, confiabilidade, usabilidade, eficiência, manutenibilidade e portabilidade.
- Definição colaborativa de requisitos de qualidade: a norma apoia

equipes ágeis na definição clara e mensurável dos requisitos de qualidade, alinhando desenvolvedores, product owners e stakeholders. • Melhoria da comunicação e transparência com o cliente: a NBR ISO/IEC 9126 fornece uma linguagem comum entre equipes de desenvolvimento e clientes, fundamental em ambientes ágeis que valorizam a colaboração. Aprimoramento contínuo do processo de desenvolvimento: a aplicação das métricas e características da norma, associada a práticas ágeis como Retrospectives e Kaizen, promove a melhoria contínua dos processos e da qualidade do software.

Os **requisitos de qualidade externa** correspondem às necessidades percebidas pelos usuários e stakeholders, refletindo a experiência de uso, o desempenho, a confiabilidade e a usabilidade esperados do produto de software.

Os **requisitos de qualidade interna** definem o nível de qualidade necessário sob a perspectiva técnica e estrutural do produto, abrangendo modelos estáticos e dinâmicos, arquitetura, documentação e código-fonte.

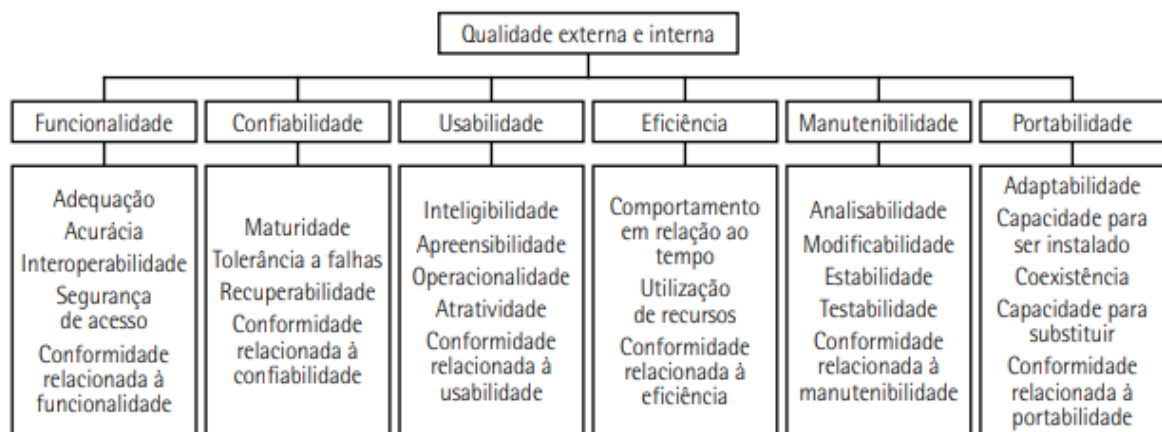


Figura 10 – Modelo de qualidade para qualidade externa e interna

A qualidade em uso representa a percepção do usuário sobre a qualidade do produto de software durante sua utilização em um contexto e cenário específicos.

O modelo de qualidade para qualidade em uso, apresentado na figura a seguir, classifica seus atributos em quatro características principais: eficácia, produtividade, segurança e satisfação.

3.3.2 NBR ISO/IEC 14598 – Avaliação do produto de software

A NBR ISO/IEC 14598 estabelece uma estrutura padronizada para a avaliação da qualidade de produtos de software, integrando-se ao modelo definido pela NBR ISO/IEC 9126. Essa norma orienta o uso de métodos de medição e avaliação, define

terminologia técnica e apresenta requisitos gerais para especificação e avaliação da qualidade.

Nas metodologias ágeis, essa norma pode ser aplicada de forma iterativa, dando suporte a CI, CD e TDD, que permitem o monitoramento constante da qualidade e o ajuste incremental dos produtos.

A ISO/IEC 14598-6, em particular, fornece módulos que especificam o modelo de qualidade: características, subcaracterísticas e métricas internas e externas, além de descrever como esses elementos devem ser aplicados e documentados em diferentes contextos.

Uso combinado das normas NBR ISO/IEC 14598 e NBR ISO/IEC 9126 cria uma base sólida para avaliar, mensurar e aprimorar continuamente a qualidade de software.

A avaliação da qualidade permite validar continuamente os requisitos de qualidade e verificar se os atributos de manutenibilidade e portabilidade estão sendo alcançados.

A **qualidade em uso** reflete o atendimento às necessidades reais dos usuários, sendo constantemente avaliada por meio de feedbacks rápidos e entregas incrementais. A **qualidade externa** é verificada durante a execução do sistema completo (hardware e software), utilizando métricas aplicadas em ambientes reais de operação, frequentemente integradas a pipelines de testes automatizados nas práticas de CI e CD. A **qualidade interna** envolve a análise de componentes estruturais como código-fonte, desempenho, banco de dados, rede e integração entre módulos, sendo monitorada por meio de ferramentas de análise estática, testes unitários e métricas de engenharia.

As métricas de qualidade interna do software servem como indicadores importantes para estimar a qualidade em uso. Quando o sistema atinge o nível de qualidade interna esperado, há maior probabilidade de que os requisitos externos e de uso também sejam plenamente atendidos.

3.3.3 NBR ISO/IEC 25000 – Guia do SQuaRE

NBR ISO/IEC 25000 trata da caracterização e medição da qualidade de produtos de software, servindo como referência central para o gerenciamento e a melhoria. Com a reorganização introduzida pela família ISO/IEC 25000, também conhecida como SQuaRE – Software Product Quality Requirements and Evaluation (Requisitos de Qualidade e

Avaliação de Produtos de Software), os temas relacionados à qualidade de software passaram a ser estruturados em cinco áreas principais.

- Requisitos de qualidade: definição e especificação das características de qualidade esperadas no produto.
- Modelo de qualidade: estrutura conceitual que descreve as características e subcaracterísticas que devem ser avaliadas.
- Avaliação: métodos e processos para medir e validar a conformidade em relação aos requisitos de qualidade.
- Gerenciamento de qualidade: práticas de planejamento, monitoramento e melhoria contínua dos processos de desenvolvimento.
- Medições: métricas e indicadores quantitativos que apoiam a análise da qualidade interna, externa e em uso.

3.4 Sistemas da qualidade: ISO 9001 e ISO 90003

A família ISO 9000 (Padrões para Gerência da Qualidade e Garantia de Qualidade) trata de sistemas de gestão da qualidade de forma geral. Ela é aplicável a qualquer tipo de organização, e não apenas à tecnologia da informação. A ISO 9000 é uma norma introdutória composta por várias outras, sendo a ISO 9001 (Modelo de Garantia de Qualidade em projeto, instalação, desenvolvimento, produção, arquitetura e serviço) a mais conhecida e utilizada.

No contexto de tecnologia da informação, a norma diretamente relacionada é a ISO/IEC 90003 (Engenharia de software), que estabelece diretrizes para a aplicação da ISO 9001 a softwares e serviços relacionados, a ISO 9001 é a principal norma internacional para sistemas de gestão da qualidade, enquanto a ISO 90003 adapta esses princípios para o setor de software.

A ISO 9001 é a norma certificável que define o que uma organização deve fazer para garantir a qualidade. Ela se baseia nos conceitos da ISO 9000, com foco na implementação prática.

ISO 9000 => Define os fundamentos e o vocabulário dos sistemas de gestão da qualidade

ISO 9001 => Estabelece os requisitos para implementar e certificar um sistema de gestão

A ISO/IEC 90003 fornece orientações específicas para a aplicação dos requisitos da ISO 9001 (Gestão da qualidade) em organizações que desenvolvem, fornecem e mantêm software. Ela não cria novos requisitos, mas interpreta a ISO 9001 no contexto de engenharia de software e serviços de TI.

3.5 Alinhamento das normas da qualidade com metodologias ágeis

Em vez de registros extensos e burocráticos, as evidências de qualidade são geradas por meio de ferramentas automatizadas, como sistemas de controle de versão, pipelines de integração contínua e relatórios de testes, a norma pode ser aplicada por meio de práticas como TDD, CI/CD e code review, que funcionam como mecanismos de verificação e validação de produto. Cada sprint ou iteração gera evidências objetivas de conformidade, substituindo a documentação tradicional por artefatos digitais rastreáveis.

As cerimônias ágeis são reuniões de planejamento, revisões e retrospectivas, que cumprem o papel das análises críticas e ações de melhoria exigidas pelas normas de qualidade. A participação ativa do cliente nas revisões garante a validação contínua do produto, enquanto as retrospectivas promovem a melhoria de processo, atendendo ao princípio da melhoria contínua (Kaizen) presente na ISO 9001.

Kaizen é uma palavra japonesa que significa “mudança para melhor”, na gestão da qualidade, Kaizen é uma filosofia de melhoria contínua, baseada em pequenas mudanças diárias e constantes, implementadas por todos na organização, e não apenas pela gerência.

Quadro 10 – Como o Kaizen é aplicado dentro do SGQ da ISO 9001

Etapa da ISO 9001	Princípio Kaizen aplicado	Exemplo prático
Planejar (Plan)	Identificar oportunidades de melhoria	Mapear gargalos em um processo de testes
Executar (Do)	Implementar pequenas mudanças	Automatizar parte das tarefas repetitivas
Verificar (Check)	Avaliar resultados obtidos	Comparar tempo de execução antes e depois
Agir (Act)	Padronizar ou ajustar melhorias	Atualizar o procedimento no manual da qualidade

4 MODELOS DA QUALIDADE DE SOFTWARE

As normas de qualidade são documentos oficiais elaborados por organizações de padronização, como a ISO ou a ABNT, com o objetivo de estabelecer requisitos e procedimentos universais que garantam a padronização e a conformidade dos processos organizacionais. As normas são regras formais e obrigatórias criadas por organizações de padronização. (Como: ISO 9001)

Os modelos de qualidade são estruturas conceituais desenvolvidas por instituições ou grupos de pesquisa que descrevem as melhores práticas e características desejáveis para avaliar ou melhorar a qualidade de produtos e processos.

Enquanto as normas orientam como fazer com qualidade e permitem certificações formais, os modelos indicam como medir e aprimorar a qualidade de forma contínua. Normas como a ISO 9001 padronizam processos de gestão, enquanto modelos como o CMMI® orientam a maturidade e a evolução da qualidade dentro das empresas.

4.1 Capability Maturity Model Integration (CMMI®)

O CMMI® é um modelo de referência desenvolvido pelo Software Engineering Institute (SEI) para ajudar organizações a avaliar e aprimorar seus processos, garantindo maior qualidade, previsibilidade e eficiência, conceito original surgiu a partir do Software Capability Maturity Model. CMMI® define cinco níveis de maturidade, que vão desde processos ad hoc até processos otimizados e continuamente melhorados:

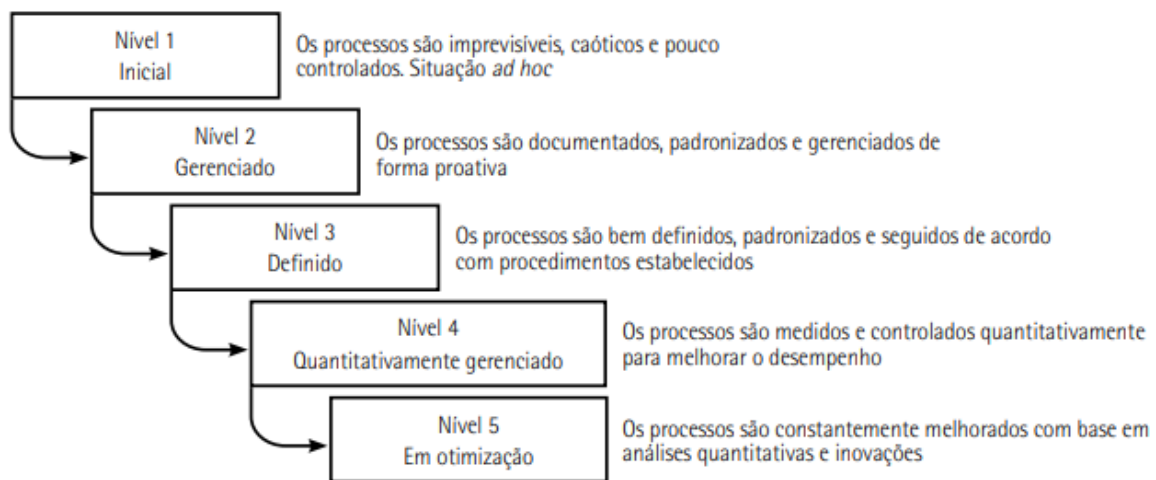


Figura 15 – Os cinco níveis de maturidade do CMMI®

CMMI® estabelece um conjunto de práticas recomendadas distribuídas em diversas áreas-chave de processo (ACP), do inglês key process areas (KPA). A integração ocorre pelo uso de ferramentas ágeis, como controle de versão, CI e testes automatizados.

Quadro 11 – Key process areas (KPA)

Gerencial Planejamento de projeto de software e gerência	Organizacional Revisão e controle pela gerência sênior	Engenharia de software Especificação, design, codificação e controle de qualidade
Nível 1 - Inicial (processo caótico - indica a fase de iniciação da implementação dos processos)		
Nível 2 - Repetitivo (processo disciplinado)		
Acompanhamento e controle de projeto Gerência de configuração Gerência de requisitos Gerência de subcontratação Medição e análise Planejamento do projeto Garantia de qualidade de processo e produto		
Nível 3 - Definido (processo padronizado e consistente)		
Análise de decisão e resolução Gerência de software integrada	Definição do processo da organização Foco no processo da organização Programa de treinamento Validação	Desenvolvimento de requisitos Gerenciamento de risco Integração do produto Solução técnica dos requisitos Verificação
Nível 4 - Gerenciado (processo previsível)		
Gerenciamento quantitativo de processos	Desempenho do processo organizacional	
Nível 5 - Otimização (melhoria contínua)		
	Gestão do processo organizacional	Análise e resolução de causas

CMMI® também pode ser aplicado em metodologias ágeis, como Scrum e XP, orientando a estruturação de práticas ágeis dentro de um framework de maturidade.

4.2 SPICE – ISO 15504

A ISO/IEC 15504, conhecida como SPICE (Software Process Improvement & Capability Determination, ou Melhoria do Processo de Software e Determinação da Capacidade), é uma norma internacional voltada à avaliação da capacidade e maturidade dos processos de desenvolvimento de software em uma organização.

A ISO/IEC 15504 estabelece um modelo estruturado de avaliação de processos, organizado em seis níveis de capacidade, que vão do Nível 0 (Incompleto) ao Nível 5 (Otimizado).

Esse modelo é composto por um conjunto de processos, duas dimensões principais: • dimensão de processo; • dimensão de capacidade.

Na dimensão de processo, o modelo organiza as atividades em cinco grandes categorias: • Cliente-fornecedor: processos que tratam da interação entre a organização e seus clientes • Engenharia: processos relacionados a especificação, projeto, construção, teste e manutenção de software e sistemas. • Suporte: processos que fornecem suporte técnico e administrativo aos demais. • Gerência:

processos que envolvem planejamento, acompanhamento, controle e medição de projetos e recursos.

- Organização: processos voltados à melhoria contínua, à definição de políticas e ao desenvolvimento de competências organizacionais.

Na dimensão da capacidade, o SPICE define seis níveis de capacitação dos processos, que representam estágios graduais de maturidade e controle. Esses níveis permitem avaliar a capacidade da organização em executar, gerenciar e aprimorar cada processo. O objetivo dessa dimensão é diagnosticar a efetividade dos processos e guiar a evolução organizacional por meio de práticas mensuráveis e estruturadas.

- Nível 0 (Incompleto): o processo não atinge seus objetivos ou é executado de forma irregular e não controlada.
- Nível 1 (Executado ou Realizado): o processo é realizado de forma ad hoc, sem uma abordagem consistente ou padronizada.
- Nível 2 (Gerenciado): o processo é planejado, monitorado e controlado de acordo com procedimentos estabelecidos.
- Nível 3 (Estabelecido): o processo é definido, documentado e padronizado em toda a organização, permitindo ações corretivas quando necessário.
- Nível 4 (Previsível): o processo é quantitativamente gerenciado, com metas, indicadores e medidas de desempenho sistematicamente aplicadas.
- Nível 5 (Otimizado): o processo é continuamente avaliado e aprimorado com base em dados, visando inovação e adaptação às mudanças organizacionais.

elementos normativos do SPICE – ISO 15504:

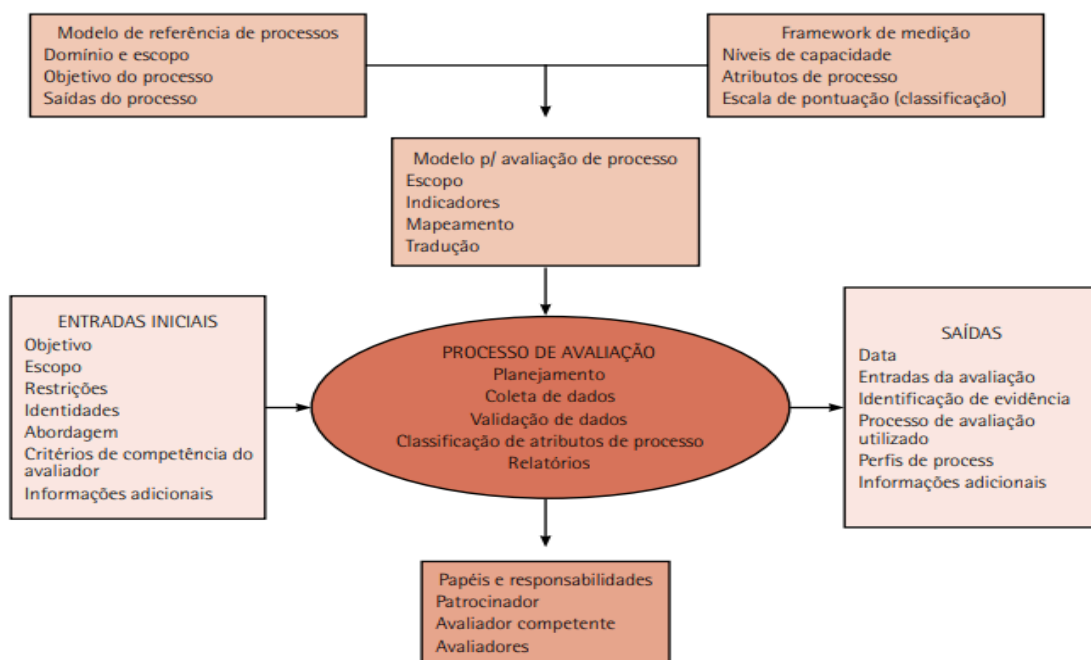


Figura 16 – Elementos normativos do SPICE – ISO 15504

Dimensão de processos do SPICE – ISO/IEC 15504: apresenta um conjunto de subprocessos e respectivas atividades que estruturam a avaliação e a melhoria dos processos organizacionais, essa dimensão organiza os processos em três grandes áreas: processos primários, processos organizacionais e processos de apoio.

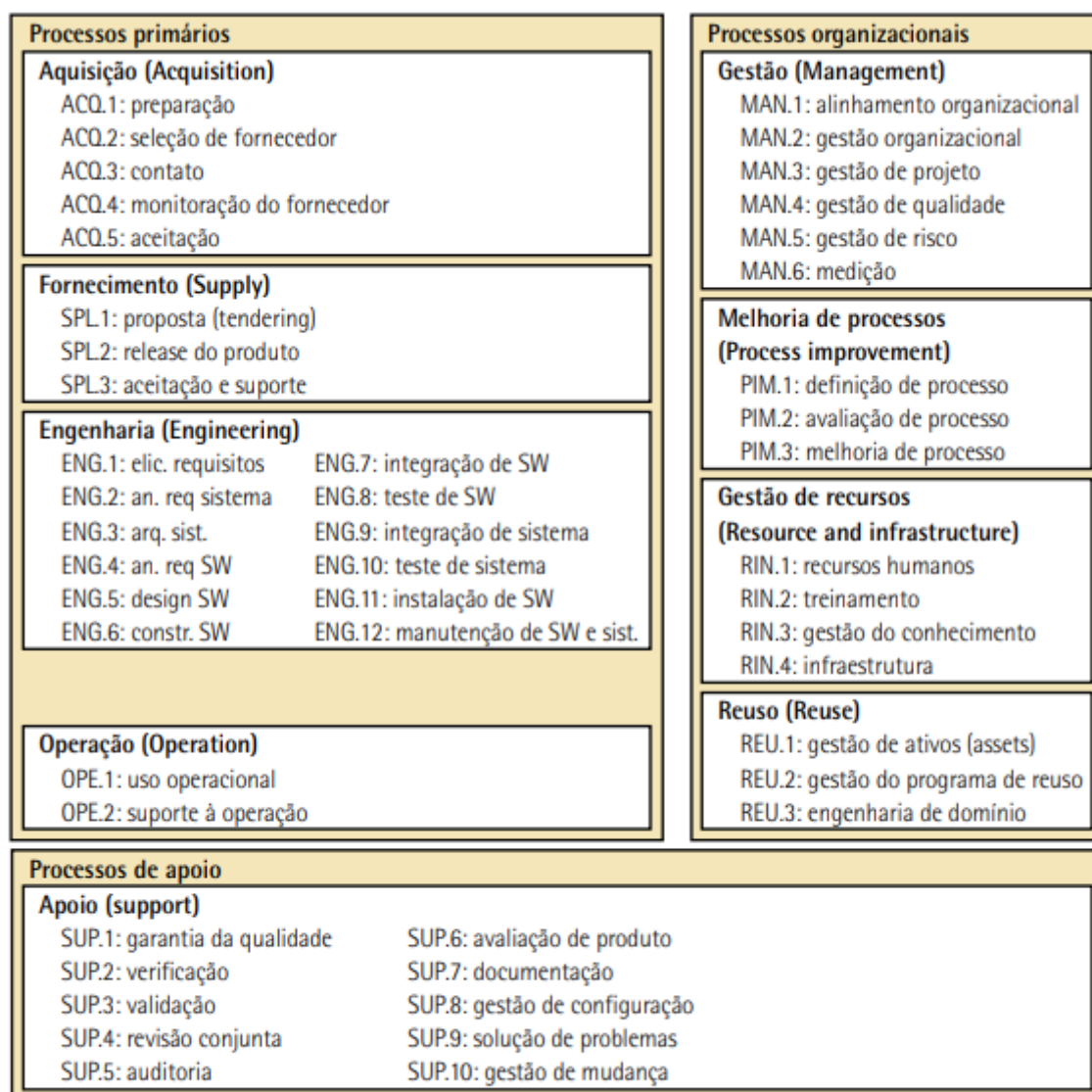


Figura 17 – Dimensão dos processos do SPICE – ISO 15504

4.3 MPS.BR

MPS.BR (Melhoria do Processo de Software Brasileiro) é um modelo de melhoria de processos voltado para o desenvolvimento e a manutenção de software, criado com o objetivo de elevar a qualidade e a competitividade da indústria nacional.

O modelo define sete níveis de maturidade, que representam o grau de institucionalização e controle dos processos de software dentro de uma organização.

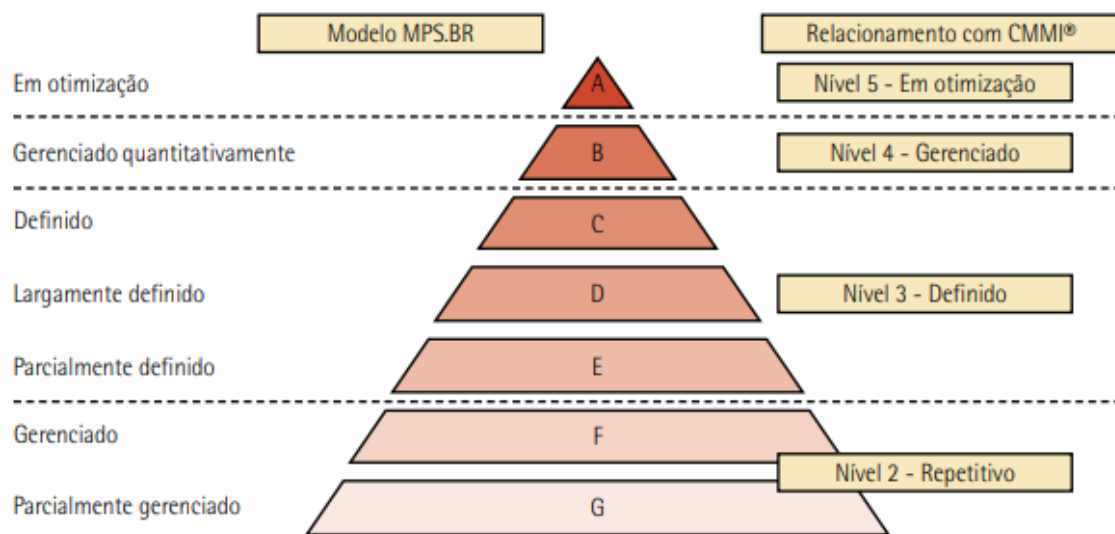


Figura 18 – Modelo MPS.BR definido em sete níveis e seus relacionamentos com CMMI®

4.3.1 Integração com metodologias ágeis

MPS.BR pode ser aplicado de forma complementar às metodologias ágeis, as metodologias ágeis priorizam entregas rápidas, feedback constante e foco no cliente, o MPS.BR fornece critérios objetivos de avaliação, padronização e medição de desempenho.

4.4 Avaliação da qualidade em projetos ágeis

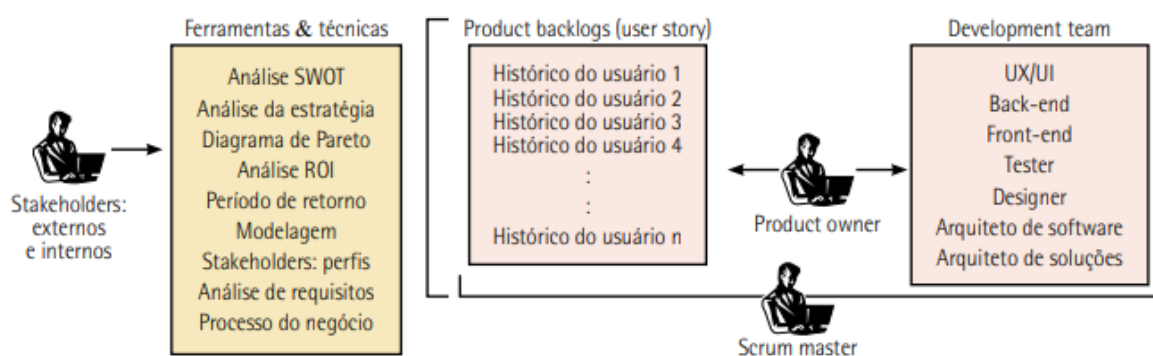


Figura 19 – Modelo SAFe® de gerenciamento Lean-Ágil de software

A avaliação da qualidade em ambientes ágeis pode ser apoiada por modelos e normas internacionais. A ISO/IEC 25000 (SQuaRE), por exemplo, define características e métricas de qualidade de produto como funcionalidade, confiabilidade, usabilidade e eficiência, que podem ser utilizadas em sprints reviews ou retrospectivas como indicadores de desempenho. Já a ISO/IEC SPICE – ISO

15504 e o CMMI® oferecem estruturas de avaliação da capacidade e maturidade dos processos, que podem ser integradas às práticas ágeis sem comprometer a flexibilidade, estabelecendo critérios objetivos de melhoria contínua.

4.5 Métodos, ferramentas e técnicas (Mf&T): metodologia para integração ágil-qualidade com análise SWOT em conformidade com [MPS.BR](#)

4.5 Métodos, ferramentas e técnicas (Mf&T): metodologia para integração ágil-qualidade com análise SWOT em conformidade com [MPS.BR](#)

Quadro 12 – Matriz SWOT

Elemento	Tradução	Tipo de fator	Significado
S – Strengths	Forças	Interno	Pontos fortes da organização ou projeto: é o que se faz bem, recursos, competências e vantagens competitivas
W – Weaknesses	Fraquezas	Interno	Pontos fracos: limitações, falhas de processo, falta de recursos ou competências
O – Opportunities	Oportunidades	Externo	Fatores externos positivos que podem ser explorados: tendências de mercado, tecnologias emergentes, novas demandas
T – Threats	Ameaças	Externo	Fatores externos que podem causar riscos ou prejuízos: concorrência, mudanças econômicas e regulações

	Fatores positivos	Fatores negativos
Fatores internos	S – Forças (strengths)	W – Fraquezas (weaknesses)
Fatores externos	O – Oportunidades (opportunities)	T – Ameaças (threats)

Unidade III

5 PROJETO E CONSTRUÇÃO DO SOFTWARE

O projeto e a construção do software representam o momento em que a visão conceitual se transforma em produto funcional.

5.1 Gerenciamento do projeto de software

O ciclo de vida do projeto é gerenciado através da execução de uma série de atividades conhecidas como processo de gerenciamento de projetos.

Um processo é um conjunto de atividades, ações e tarefas realizadas na criação de algum artefato.

Todo processo deve ser claramente delimitado, definindo explicitamente seu ponto de início (input) e seu ponto de término (output).

- Input (entradas): representa o ponto inicial do processo, correspondendo aos requisitos e aos recursos necessários para iniciar uma atividade ou ciclo de desenvolvimento. Esses itens, em uma forma ampla, referem-se aos requisitos de entrada do processo, tais como: objetos claros e específicos, recursos humanos necessários, materiais, tecnologia a ser empregada, fornecedores, componentes do software/sistema, metodologias, ferramentas, custo, cronograma, procedimentos de trabalho e documentos.
- Processo (atividades e tarefas): é a combinação estruturada e iterativa que organiza os itens de entrada para criar um produto de software ou de sistema, ou um componente de outro produto, ou para executar um determinado trabalho. Por meio de práticas ágeis e ferramentas colaborativas, o processo envolve a transformação contínua dos requisitos em incrementos de software funcional, a partir de sprints, integração e entrega contínuas, testes automatizados, revisões de código e feedback constante do cliente. Dessa forma, o processo torna-se dinâmico, adaptativo e centrado na geração de valor.
- Output (saídas): são os resultados finais gerados pelo processo. Corresponde à entrega validada do produto, componente ou serviço, conforme os critérios de aceitação definidos no processo. Novos registros são feitos e novas medições e verificações são realizadas para confrontar o resultado obtido com o objetivo planejado para o processo. No encerramento do processo, são realizadas medições, verificações e registros que comparam os resultados obtidos com os objetivos planejados, promovendo o aprendizado organizacional e a melhoria contínua.

Quadro 13 – Arcabouço do processo com métodos, ferramentas e técnicas (MF&T)

Métodos	Ferramentas	Técnicas
Cascata	Astah	
Incremental	Azure DevOps	
Espiral	CASE	Análise de documentos
Scrum	Jira	User stories
XP	Trello	Prototipagem
Kanban	Figma	Modelagem UML
FDD	Android Studio	Arquiteturas
SAFe®	Eclipse	Padrões de projeto
DevOps	NetBeans	Técnicas de refatoração
SOA	PyCharm	Testes
ISO 12207	Visual Studio	Qualidade, PDCA
ISO 25000	GitHub	Matriz SWOT
CMMI®	Microsoft Teams	Versionamento
MPS.BR	Microsoft Project	
	Google Cloud Platform	

Arcabouço do processo é como uma “caixa de ferramentas” com recursos para projetar e construir software, em que a equipe desenvolvedora se baseia para organizar as atividades, definir práticas recomendadas e orientar o uso de tecnologias de apoio.

5.2 Guia do conhecimento em gerenciamento de projetos PMBOK®

O Guia PMBOK® – Project Management Body of Knowledge (Guia do Conhecimento em Gerenciamento de Projetos) representa o principal modelo de referência internacional, justamente, para o Gerenciamento de Projetos, consolida um conjunto de práticas, processos, ferramentas e técnicas reconhecidas como boas práticas no gerenciamento de projetos. Ele define uma estrutura conceitual que abrange áreas de conhecimento (como escopo, tempo, custo, qualidade, riscos, comunicações e integração) e grupos de processos (iniciação, planejamento, execução, monitoramento e controle, e encerramento), que podem ser adaptados conforme o contexto organizacional e a metodologia utilizada.

A 6ª edição do PMBOK®, publicada em 2017, consolidou uma visão estruturada baseada em 5 grupos de processos e 10 áreas de conhecimento, oferecendo um modelo sistemático para planejar e controlar projetos de maneira sequencial e preditiva. A 7ª edição do PMBOK® (2021) introduziu uma mudança significativa: em

vez de focar em processos fixos, passou a estruturar-se em 12 princípios de gerenciamento de projetos e 8 domínios de desempenho.

Importância do gerenciamento de projetos

O gerenciamento de projetos oferece uma abordagem estruturada, sistemática e orientada a resultados para o planejamento, execução, monitoramento e controle de projetos.

- Planejamento e organização: fornece um framework sistemático para planejar, executar e controlar projetos de forma eficiente, estabelecendo objetivos claros, escopo definido, cronogramas realistas.
- Gestão de riscos: possibilita a identificação precoce e gestão proativa de riscos, análise e mitigação de riscos.
- Comunicação e alinhamento: promove uma comunicação eficaz e colaborativa entre as equipes de desenvolvimento, stakeholders e clientes.
- Controle e melhoria contínua: ao manter o projeto dentro do escopo, do prazo e do orçamento planejado, o gerenciamento de projetos aumenta a taxa de sucesso e a satisfação do cliente.

Quadro 14 – Componentes-chave do Guia PMBOK®

Ciclo de vida do projeto
Fase do projeto
Revisão de fase
Grupo de processos de gerenciamento de projetos
Processo de gerenciamento de projetos
Área de conhecimento em gerenciamento de projetos

Fonte: PMBOK® (2017).

- Grupo de processos de iniciação: tem como objetivo definir um novo projeto ou uma nova fase de um projeto existente.
- Grupo de processos de planejamento: envolve o desenvolvimento de um plano detalhado que estabelece os objetivos, o escopo, as atividades, os recursos e o cronograma do projeto.
- Grupo de processos de execução: corresponde à implementação do que foi planejado, com a realização das atividades necessárias para atingir os objetivos do projeto.
- Grupo de processos de monitoramento e controle: constituem uma fase essencial do gerenciamento de projetos, voltada ao acompanhamento do desempenho, controle das atividades e gestão de mudanças.
- Grupo de processos de encerramento: marcam a conclusão formal do projeto ou de uma fase. Nessa etapa, as entregas são verificadas e aceitas pelos stakeholders, contratos são encerrados e recursos são liberados.

Além dos grupos de processos, o PMBOK® também organiza o gerenciamento de projetos em áreas de conhecimento.

Quadro 15 – Grupo de processos de gerenciamento de projetos e mapeamento das áreas de conhecimento

Áreas de conhecimento	Grupos de processos de gerenciamento de projetos				
	Grupo de processos de iniciação	Grupo de processos de planejamento	Grupo de processos de execução	Grupo de processos de monitoramento e controle	Grupo de processos de encerramento
4. Gerenciamento da integração do projeto	4.1 Desenvolver o termo de abertura do projeto	4.2 Desenvolver o Plano de Gerenciamento do Projeto	4.3 Orientar e gerenciar o trabalho do projeto 4.4 Gerenciar o conhecimento do projeto	4.5 Monitorar e controlar o trabalho do projeto 4.6 Realizar o controle integrado de mudanças	4.7 Encerrar o projeto ou fase
5. Gerenciamento do escopo do projeto		5.1 Planejar o gerenciamento do escopo 5.2 Coletar os requisitos 5.3 Definir o escopo 5.4 Criar a EAP		5.5 Validar o escopo 5.6 Controlar o escopo	

Áreas de conhecimento	Grupos de processos de gerenciamento de projetos				
	Grupo de processos de iniciação	Grupo de processos de planejamento	Grupo de processos de execução	Grupo de processos de monitoramento e controle	Grupo de processos de encerramento
6. Gerenciamento do cronograma do projeto		6.1 Planejar o gerenciamento do cronograma 6.2 Definir as atividades 6.3 Sequenciar as atividades 6.4 Estimar as durações das atividades 6.5 Desenvolver o cronograma		6.6 Controlar o cronograma	
7. Gerenciamento dos custos do projeto		7.1 Planejar o gerenciamento dos custos 7.2 Estimar os custos 7.3 Determinar o orçamento		7.4 Controlar os custos	
8. Gerenciamento da qualidade do projeto		8.1 Planejar o gerenciamento da qualidade	8.2 Gerenciar a qualidade	8.3 Controlar a qualidade	
9. Gerenciamento dos recursos do projeto		9.1 Planejar o gerenciamento dos recursos 9.2 Estimar os recursos das atividades	9.3 Adquirir recursos 9.4 Desenvolver a equipe 9.5 Gerenciar a equipe	9.6 Controlar os recursos	
10. Gerenciamento das comunicações do projeto		10.1 Planejar o gerenciamento das comunicações	10.2 Gerenciar as comunicações	10.3 Monitorar as comunicações	
11. Gerenciamento dos riscos do projeto		11.1 Planejar o gerenciamento dos riscos 11.2 Identificar os riscos 11.3 Realizar a análise qualitativa dos riscos 11.4 Realizar a análise quantitativa dos riscos 11.5 Planejar as respostas aos riscos	11.6 Implementar respostas aos riscos	11.7 Monitorar os riscos	
12. Gerenciamento das aquisições do projeto		12.1 Planejar o gerenciamento das aquisições	12.2 Conduzir as aquisições	12.3 Controlar as aquisições	
13. Gerenciamento das partes interessadas do projeto	13.1 Identificar as partes interessadas	13.2 Planejar o engajamento das partes interessadas	13.3 Gerenciar o engajamento das partes interessadas	13.4 Monitorar o engajamento das partes interessadas	

Adaptado de: PMBOK® (2017).

5.2.1 Abordagens adaptativas e metodologias ágeis no gerenciamento de projetos

A integração do PMBOK® com abordagens ágeis levou à 7ª edição do Guia PMBOK® (2021), passou a estruturar-se em 12 princípios de gerenciamento de projetos e 8 domínios de desempenho.

Os 12 princípios de gerenciamento de projetos do PMBOK® 7ª edição:

1)Seja um profissional diligente, respeitoso e atencioso: o gerente de projetos deve agir com integridade, responsabilidade e ética, valorizando as pessoas e os recursos. 2)Crie um ambiente colaborativo para a equipe: promover uma cultura de confiança, respeito e engajamento é essencial. 3)Envolva-se de fato com as partes interessadas: as partes interessadas (stakeholders) devem ser continuamente identificadas, engajadas e ouvidas. 4)Enfoque no valor: todo projeto deve gerar valor mensurável para as partes interessadas. 5)Reconheça, avalie e reaja às interações do sistema: os projetos operam dentro de sistemas dinâmicos (organizacionais, sociais e tecnológicos). 6)Demonstre comportamentos de liderança: o ambiente de projetos envolve riscos, variabilidade e interdependências. 7) Faça a adaptação de acordo com o contexto: cada projeto é único. As práticas e métodos devem ser personalizadas de acordo com o tamanho, complexidade, riscos e ambiente do projeto. 8)Inclua qualidade nos processos e nas entregas: a qualidade deve ser planejada e incorporada desde o início, não apenas verificada no final. 9)Complexidade: os projetos operam em ambientes complexos e dinâmicos, exigindo do gerente a capacidade de compreender, lidar e aprender com essa complexidade. 10) Otimize as respostas aos riscos: gerenciar riscos é um processo proativo e contínuo. Envolve identificar oportunidades e ameaças. 11) Adote a capacidade de adaptação e resiliência: projetos enfrentam mudanças e imprevistos e devem ser flexíveis, com aprendizado rápido e ajustado a novas condições. 12) Aceite a mudança para alcançar o futuro estado previsto: o sucesso de um projeto não se resume à entrega pontual. Ele deve gerar benefícios duradouros.

8 domínios de desempenho do PMBOK® 7ª edição:

1)Partes interessadas (stakeholders): foca na identificação, análise, engajamento e comunicação com as partes interessadas ao longo de todo o ciclo de vida do projeto. 2)Equipe (team): trata da formação, desenvolvimento e liderança da equipe do projeto. Envolve promover colaboração, confiança, diversidade e empoderamento.

3)Abordagem de desenvolvimento e do ciclo de vida (development approach and life cycle): refere-se à escolha e aplicação da abordagem mais adequada (preditiva, adaptativa ou híbrida) e à definição do ciclo de vida do projeto (fases, marcos e entregas). 4)Planejamento (planning): envolve o planejamento contínuo e adaptável do projeto, considerando objetivos, escopo, cronograma, recursos, riscos e comunicações. 5) Trabalho do projeto (project work): abrange a execução, coordenação e monitoramento das atividades que produzem as entregas. 6)Entrega (delivery): foca na entrega de resultados e benefícios que atendam às expectativas das partes interessadas e gerem valor real para a organização. 7)Medição (measurement): trata da avaliação do desempenho do projeto por meio de métricas, indicadores e informações quantitativas e qualitativas. 8)Incerteza (uncertainty): envolve a identificação e o gerenciamento proativo de riscos, oportunidades e variabilidades.

5.3 Guia de referência para engenharia de software SWEBOK®

O Software Engineering Body of Knowledge (SWEBOK®), traduzido como Corpo de Conhecimento em Engenharia de Software, é um guia internacional que consolida o conhecimento essencial da engenharia de software.

Ele foi desenvolvido sob a coordenação do IEEE Computer Society, com apoio do ISO/IEC, e tem como objetivo principal definir o escopo e o corpo de conhecimento que todo engenheiro de software deve dominar, tem por finalidade servir de referência global para a formação, certificação e prática profissional em engenharia de software. Ele não dita como o trabalho deve ser feito, mas define o que deve ser conhecido e compreendido.

Seu propósito é: • padronizar a terminologia e os conceitos fundamentais da área; • definir competências essenciais para engenheiros de software; • orientar currículos acadêmicos, programas de certificação e avaliações de competência; • promover a uniformidade e o reconhecimento da engenharia de software como uma disciplina de engenharia legítima.

As 18 áreas de conhecimento (KAs) do SWEBOK® v4.0 são:

1) Requisitos de software (software requirements): trata da identificação, análise, especificação, validação e gestão dos requisitos de software. 2) Arquitetura de software (software architecture): aborda o projeto e a estrutura de alto nível de um sistema, incluindo componentes, interações e decisões arquiteturais que impactam desempenho, segurança e manutenção. 3) Projeto de software (software design): foco na transformação dos requisitos em uma solução detalhada, especificando componentes, algoritmos, interfaces e dados. 4) Construção de software (software construction): compreende as atividades de codificação, verificação unitária e integração, aplicando boas práticas de programação, padrões de código e ferramentas de desenvolvimento. 5) Teste de software (software testing): engloba as técnicas e processos usados para avaliar a qualidade e verificar se o software atende aos requisitos. 6) Operações de engenharia de software (software engineering operations): trata das atividades de implantação, operação e suporte do software em ambiente de produção. 7) Manutenção de software (software maintenance): abrange as atividades pós-entrega, como correção de defeitos, melhorias e adaptações do software para novos ambientes ou necessidades do usuário. 8) Gerenciamento de configuração de software (software configuration management): responsável pelo controle de versões, rastreabilidade e integridade dos artefatos do projeto, garantindo que alterações sejam gerenciadas de forma sistemática. 9) Gerenciamento de engenharia de software (software engineering management): envolve o planejamento, acompanhamento e controle de projetos de software, incluindo escopo, cronograma, custos, riscos e qualidade, para garantir o sucesso da entrega. 10) Processo de engenharia de software (software engineering process): define e melhora os processos utilizados para desenvolver e manter software, promovendo padronização, eficiência e melhoria contínua organizacional. 11) Modelos e métodos de engenharia de software (software engineering models and methods): inclui os modelos de ciclo de vida (como cascata, incremental e ágil) e os métodos formais ou empíricos aplicados ao desenvolvimento e análise de software. 12) Qualidade de software (software quality): foca na garantia e avaliação da qualidade, cobrindo normas, métricas, revisões e auditorias que asseguram que o produto atenda às expectativas e requisitos definidos. 13) Segurança de software (software security): trata da proteção do software contra vulnerabilidades, ataques e falhas, incorporando princípios de segurança desde o projeto até a operação. 14) Prática profissional em engenharia de software (software engineering professional

practice): aborda a ética, comunicação, trabalho em equipe, responsabilidade social e competências profissionais necessárias para a prática responsável da engenharia de software. 15) Economia da engenharia de software (software engineering economics): analisa o impacto econômico das decisões de engenharia, considerando custos, benefícios, riscos e trade-offs (relação de compromisso) ao longo do ciclo de vida do software. 16) Fundamentos de computação (computing foundations): reúnem os conceitos básicos de ciência da computação como estruturas de dados, algoritmos, arquitetura de computadores e sistemas operacionais, que sustentam a engenharia de software. 17) Fundamentos matemáticos (mathematical foundations): inclui os conceitos matemáticos e lógicos aplicados à modelagem, análise e verificação de software, como álgebra, estatística, lógica formal e teoria dos grafos. 18) Fundamentos de engenharia (engineering foundations): apresenta os princípios gerais da engenharia, como análise de sistemas, modelagem, controle de qualidade, segurança e ética, aplicados ao contexto do desenvolvimento de software.

5.3.1 Panorama dos padrões de engenharia de software

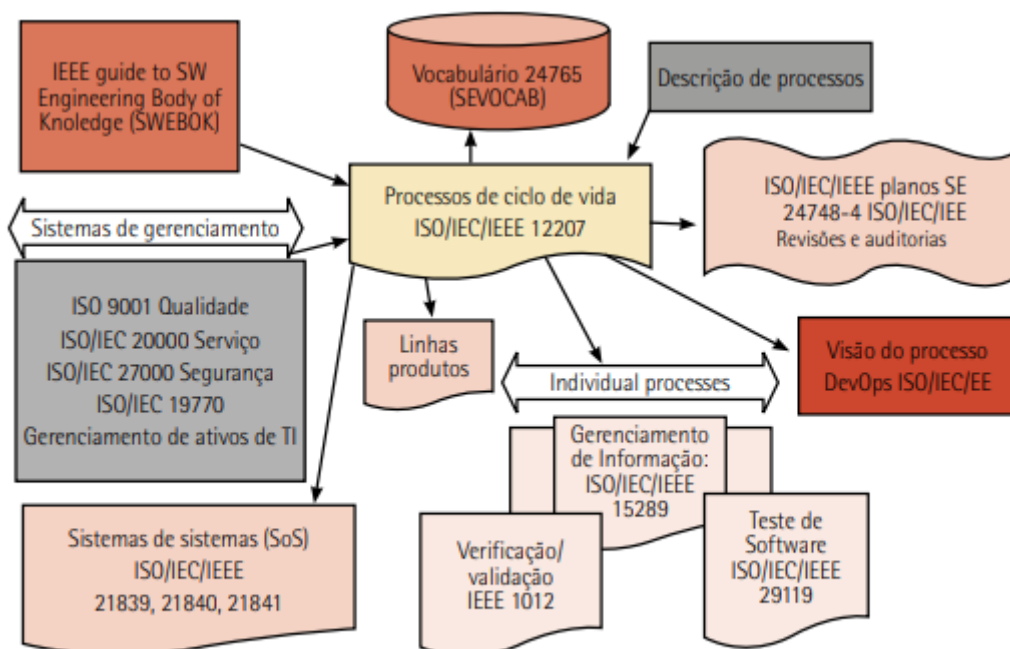


Figura 28 – Panorama dos padrões de engenharia de software

Existem padrões sobre como planejar e gerenciar a engenharia de software (ISO/IEC/IEEE 24748-5) e como conduzir revisões e auditorias rigorosas, apropriadas para softwares críticos como sistemas aeroespaciais e de defesa

(ISO/IEC/IEEE 24748-8). Além disso, existem muitos outros padrões especializados cobrindo processos individuais e abordagens modernas aos processos, como o ISO/IEC/IEEE 32675 para DevOps, bem como o IEEE 1012 para verificação/validação e o ISO/IEC/IEEE 29119 para teste de software (em múltiplas partes).

As séries mais especializadas se concentram em processos e ferramentas para engenharia de linha de produto e para sistemas de sistemas (SoS). Os padrões de sistema de sistemas ISO/IEC/IEEE 21839, 21840 e 21841 explicam como usar processos de engenharia de sistemas quando o sistema de interesse (SOI) é uma parte constituinte de um sistema de sistemas. • ISO 9001 para gestão da qualidade (veja na unidade II o tópico 3.4 Sistemas da qualidade: ISO 9001 e ISO 90003); • ISO/IEC 20000 para gestão de serviços; • a série ISO/IEC 27000 para gestão de segurança da informação; • a série ISO/IEC 19770 para gestão de ativos de TI (como hardware e software).

5.4 Arquiteturas de sistemas: uma abordagem integrada entre PMBOK® e SWEBOK®

No planejamento estratégico de projetos (abordado pelo PMBOK®), a arquitetura de sistemas atua como uma ponte para a execução técnica da engenharia de software (abordada pelo SWEBOK®). O PMBOK® é o guia de boas práticas para o gerenciamento de projetos. Na arquitetura de sistemas, ele lida desde a concepção, o controle do ciclo de desenvolvimento, a entrega do produto software (deliverable) e o fator de risco/qualidade

Principais áreas de conhecimento aplicadas:

- Gerenciamento de integração do projeto: a arquitetura fornece o plano mestre técnico que integra o trabalho de diferentes equipes
- Gerenciamento do escopo do projeto: a arquitetura é um dos principais produtos de entrega do projeto de software
- Gerenciamento da qualidade do projeto: as decisões arquiteturais afetam diretamente a qualidade do produto final. Atributos como escalabilidade, desempenho e segurança (que são definidos na arquitetura) são verificados e validados por meio de processos de qualidade do PMBOK®.
- Gerenciamento dos riscos do projeto: a inadequação ou a não documentação da arquitetura é um risco

significativo para o projeto. O PMBOK® exige que o gerente de projetos identifique, analise e planeje respostas para riscos arquiteturais

O SWEBOK: • Área de conhecimento “projeto de software” (software design): a arquitetura é o componente central dessa área. • Definição de estrutura: o SWEBOK® aborda estilos arquiteturais (como cliente-servidor, camadas, microsserviços), padrões e a tomada de decisões sobre a estrutura que garantirá os atributos de qualidade (como desempenho, segurança e manutenibilidade).

A arquitetura de sistemas surge como um elo entre gestão e engenharia, sendo responsável por estruturar os componentes, definir padrões de interação e garantir atributos de qualidade como desempenho, escalabilidade e manutenibilidade.

5.4.1 Arquitetura de sistemas: elementos, tipos e impactos na qualidade

Arquiteturas de sistemas são estruturas organizacionais que definem como os componentes de um sistema interagem, se comunicam e funcionam em conjunto para atender aos requisitos de negócio e técnicos. Ela descreve a estrutura, comportamento e relações entre os elementos de um sistema, como módulos, serviços, interfaces e protocolos.

A arquitetura de sistemas é o projeto estrutural que sustenta o ambiente computacional, englobando a definição dos componentes principais, suas propriedades visíveis externamente e os relacionamentos entre eles.

Os principais elementos da arquitetura de sistemas são:

- **Camadas:** separação lógica entre apresentação, lógica de negócio e integração (ou dados), com foco no negócio. Veja a figura a seguir:

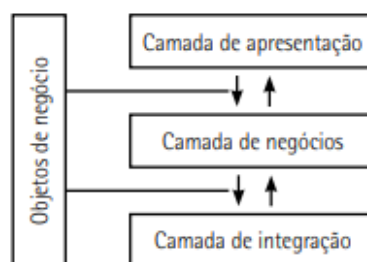


Figura 29 – Arquitetura em camadas

Fonte: Versolatto (2015).

- **Serviços:** funcionalidades específicas que o sistema de software oferece.
- **Interfaces:** pontos de interação entre componentes ou com o usuário.
- **Protocolos de comunicação:** regras, normalmente de rede de computadores para troca de dados entre os módulos do sistema de software.
- **Padrões de design:** soluções reutilizáveis para problemas recorrentes.

Já os tipos de arquitetura de sistemas são:

- Monolítico: todos os componentes integrados em um único bloco, servindo como base para a maioria dos sistemas no início da computação moderna

Exemplo de arquitetura monolítica para atendimento de terminais como PDV, leitores de códigos de barras, controle de catracas, sinalizadores:

- Cliente-servidor: separação entre quem solicita e quem fornece os serviços, estabelecendo o primeiro grande padrão de distribuição básica e separando a apresentação (cliente) da lógica e dos dados (servidor).
- Arquitetura em camadas: organização hierárquica por responsabilidades com o foco em resolver problemas de manutenibilidade e organização dentro de um sistema de software, que pode ser também um monólito.
- Arquitetura orientada a serviços (SOA): são componentes de serviços independentes (componentização), introduzindo a ideia de grandes serviços reusáveis e permitindo o desacoplamento e a integração entre sistemas corporativos grandes e heterogêneos.
- Microserviços: divisão em pequenos serviços autônomos e escaláveis, sendo uma evolução direta do SOA com foco em serviços menores, independentes, autônomos (com seus próprios bancos de dados e módulos de implantação) e na comunicação leve (geralmente HTTP/REST ou envio de mensagens). O foco está na velocidade de disponibilidade e escalabilidade individual. A arquitetura de microserviços é, atualmente, a arquitetura projetada mais alinhada com os princípios de sistemas distribuídos.

6 ECOSSISTEMAS DIGITAIS E ENGENHARIA DE SOFTWARE CONECTADA

Os ecossistemas digitais representam esse novo ambiente de criação e evolução tecnológica, no qual múltiplas plataformas, organizações, usuários e tecnologias interagem de forma contínua.

6.1 Compreendendo o ecossistema e o negócio digital

Nos modelos tradicionais, o software atende a um conjunto fechado de requisitos de um cliente específico. Já os ecossistemas digitais operam como redes abertas, nas quais múltiplos atores, empresas, usuários, desenvolvedores, provedores de serviços e plataformas tecnológicas interagem, criando valor de forma contínua.

Em resumo, o ecossistema digital é formado por um conjunto de elementos interdependentes que trabalham juntos para criar, entregar, integrar e evoluir

produtos e serviços digitais. Esses elementos não existem isoladamente: eles cooperam, competem e se influenciam

Os principais componentes de um ecossistema digital moderno são:

- Plataformas e infraestruturas tecnológicas: é a base sobre a qual tudo funciona. Incluem: — sistemas operacionais (Android, iOS, Windows); — plataformas de nuvem (AWS, Azure, Google Cloud); — marketplaces e hubs de integração (Salesforce, Shopify). Nessas plataformas são estabelecidos padrões, APIs, regras e protocolos que orientam a inovação.
- Comunidades de desenvolvedores e parceiros: são as pessoas e organizações que cocriam soluções dentro do ecossistema, como: — desenvolvedores independentes; — empresas terceiras; — startups; — fornecedores de serviços complementares.
- Usuários e consumidores: é o centro do ecossistema digital. Incluem: — usuários finais; — empresas clientes; — organizações públicas ou privadas
- Serviços digitais, APIs e módulos interoperáveis: conjunto de funcionalidades que possibilitam comunicação e composição entre sistemas, como: — APIs abertas; — microsserviços; — web services; — gateways e middlewares.
- Regras de governança e normas: conjunto de diretrizes que mantêm o ecossistema digital coerente e funcional: — políticas de uso e privacidade; — licenças de software; — standards (REST, OAuth, OpenAPI); — boas práticas de comunidade.
- Dados e fluxos de informação: é o “combustível” do ecossistema digital como: — dados de usuários; — dados de operação; — telemetria;
- Ferramentas de desenvolvimento e suporte: um ecossistema digital é robusto e inúmeras ferramentas de desenvolvimento e suporte oferecem recursos para facilitar a criação e evolução de soluções: — SDKs e frameworks; — IDEs e repositórios; — DevOps, CI/CD, pipelines automatizados; — documentação, tutoriais e fóruns.
- Comunidade de suporte, feedback e inovação: é formada por: — fóruns; — grupos de usuários; — comunidades open source; — eventos e hackathons.
- Mecanismos de monetização e sustentabilidade: permitem que o ecossistema digital se mantenha: — assinaturas, licenciamento, pay-per-use; — marketplaces que compartilham receita; — incentivos para desenvolvedores.

6.2 Gestão do desenvolvimento ágil em ecossistemas digitais

A gestão ágil atua como um mecanismo de alinhamento, permitindo que informações fluam com transparência e que decisões sejam tomadas com base em dados, métricas e validação contínua. Práticas como backlog compartilhado, definição clara de responsabilidades e cadências de sincronização entre equipes são determinantes para garantir coerência entre as diversas partes envolvidas.

6.3 Ecossistemas distribuídos e inteligentes

Esses ecossistemas são compostos por múltiplos serviços, dispositivos e agentes que operam de forma colaborativa, muitas vezes em tempo real, sobre infraestruturas escaláveis e heterogêneas. Tecnologias como computação em nuvem, microsserviços, APIs, inteligência artificial e Internet das Coisas (IoT) convergem para formar ambientes dinâmicos, resilientes e orientados a dados.

As **categorias de multiprocessamento em sistemas distribuídos** ocorrem de duas formas:

- **Computadores multiprocessados (multiprocessamento local):** trata-se de um único computador com múltiplos processadores (ou núcleos) que compartilham memória e recursos.
- **Sistemas distribuídos em rede (multiprocessamento distribuído):** em sistemas distribuídos, cada computador executa uma aplicação específica; integrados, formam um único ambiente operacional controlado pelo sistema distribuído. É um conjunto de computadores autônomos interligados por rede, operando como um sistema único. Suas aplicações se baseiam em sistemas corporativos e web, computação em nuvem, serviços baseados em microsserviços e APIs. Suas principais características são: comunicação via rede, execução distribuída e paralela, tolerância a falhas e escalabilidade horizontal.

As aplicações atuais são concebidas como conjuntos de serviços independentes, conectados por interfaces bem definidas, que podem ser atualizados, ampliados ou substituídos sem comprometer o funcionamento global do sistema. Uma característica fundamental dos sistemas distribuídos é que os usuários pensam o sistema como um único computador.

O módulo do middleware responsável por essas operações é chamado de object request broker (ORB), que traduzido do inglês significa corretor de solicitação de

objetos. Esse módulo é responsável pela conversão dinâmica dos objetos entre os sistemas operacionais.

Os principais padrões para computação distribuída que utilizam a tecnologia ORB são: • Common Object Request Broker Architecture (CORBA); • Distributed Component Object Model (DCOM); • Java Remote Method Invocation (Java RMI).

Os dois padrões aceitos para todas as arquiteturas de objetos distribuídos são o CORBA e o DCOM. O padrão Java RMI é voltado para aplicações específicas.

Uma forma prática de implementar o multiprocessamento distribuído é o uso de cluster de computadores, em que vários computadores (nós) trabalham juntos para executar tarefas como se fossem um único sistema. Cada nó pode executar uma parte da aplicação, e o sistema distribuído coordena a execução, o balanceamento de carga e a comunicação entre os nós. Em um cluster como sistema distribuído, cada nó é um computador independente que executa aplicações específicas e colaborativas, sendo gerenciados por sistemas como Kubernetes, Hadoop ou Spark.

6.3.2 Service-oriented architecture (SOA)

A arquitetura orientada a serviços (SOA) é um marco nessa evolução ao introduzir o conceito de modularidade baseada em serviços reutilizáveis. Em uma SOA, funcionalidades específicas são expostas como serviços interoperáveis, que podem ser invocados por diferentes aplicações, plataformas ou parceiros de negócio. SOA é um modelo de desenvolvimento que organiza sistemas em serviços independentes, reutilizáveis e interoperáveis, promovendo flexibilidade e escalabilidade em ambientes distribuídos.

SOA contribui diretamente para atributos de qualidade: • Manutenibilidade: serviços desacoplados facilitam atualizações e correções pontuais. • Reusabilidade: componentes podem ser reaproveitados em diferentes contextos. • Escalabilidade: serviços podem ser replicados ou distribuídos conforme a demanda. • Confiabilidade: isolamento de falhas e monitoramento por serviço aumentando a robustez.

SOA em arquiteturas distribuídas atua como base para: • Integração de sistemas heterogêneos: via protocolos padronizados (SOAP, REST, gRPC). • Orquestração de serviços: em ambientes como nuvem, clusters e microsserviços. • Gerência de comunicação: entre serviços por barramentos (Enterprise Service Bus – ESB) ou

mensageria. • Distribuição geográfica: de componentes, mantendo consistência e desempenho

A compatibilidade entre SOA e métodos ágeis ocorre por: • Entrega incremental: serviços podem ser desenvolvidos e implantados em ciclos curtos. • Autonomia de equipes: cada team pode focar em um serviço específico. • Testabilidade: serviços isolados permitem testes automatizados e contínuos. • Adaptação rápida: mudanças nos requisitos são absorvidas com menor impacto sistêmico.

Princípios da SOA são baseados em diretrizes fundamentais que orientam o design e o desenvolvimento e integração de serviços em sistemas distribuídos. Princípios fundamentais do SOA: Serviços como unidades de funcionalidade: cada serviço representa uma função de negócio bem definida, como processar pagamento ou consultar estoque. • Baixo acoplamento: os serviços são independentes entre si, permitindo que mudanças em um não afetem os demais. • Alta coesão: cada serviço é focado em uma única responsabilidade, o que facilita manutenção e reuso. • Interoperabilidade: os serviços devem ser acessíveis por diferentes plataformas, linguagens e sistemas, geralmente via padrões abertos (por exemplo: XML, JSON, HTTP, SOAP e REST). • Reusabilidade: serviços são projetados para serem reutilizados em múltiplos contextos, evitando duplicação de lógica. • Abstração: a implementação interna do serviço é escondida do consumidor; o que importa é a interface pública. • Descoberta e localização: serviços podem ser registrados e descobertos dinamicamente por meio de catálogos ou repositórios (por exemplo: UDDI). • Contratos bem definidos: cada serviço expõe um contrato formal (por exemplo: WSDL, OpenAPI) que descreve como ele deve ser utilizado. • Autonomia: os serviços controlam seus próprios recursos e lógica, podendo ser implantados e escalados de forma independente. • Composição: serviços podem ser combinados para formar processos de negócio mais complexos, promovendo orquestração e flexibilidade.

6.3.3 Application programming interfaces (APIs)

As APIs são as principais formas de exposição de funcionalidades na web, permitindo que aplicações interajam com plataformas, bancos de dados, dispositivos e serviços externos de maneira padronizada e segura.

• REST (Representational State Transfer): estilo arquitetural baseado em recursos acessados por URLs, utilizando protocolos HTTP como GET, POST, PUT e DELETE

para operações, em que cada recurso tem um endpoint fixo, como /usuarios, /produtos. As respostas em geral são em JSON ou XML. É usado para implementar autenticação e autorização robustas como OAuth e monitorar o desempenho dos serviços expostos. • OAuth (Open Authorization): é o protocolo de autorização que permite que um aplicativo acesse recursos de outro serviço em nome do usuário, sem expor suas credenciais. Muito usado em integrações com serviços como Google, Facebook, GitHub e outros, funciona com tokens de acesso temporários que representam permissões concedidas.

6.4 Customização de serviços em ecossistemas digitais

Essa customização ocorre em múltiplas camadas, que vão desde a parametrização de APIs e configuração de endpoints (local digital onde se hospeda a API) até a personalização da lógica de negócio e da experiência do usuário.

Essas ferramentas servem para gerenciar a complexidade de aplicações distribuídas, normalmente baseadas em microsserviços, automatizando tarefas de rede, segurança e infraestrutura. Essas ferramentas incluem:

- API gateways: componente de software que atua como único ponto de entrada para um conjunto de serviços backend (frequentemente microsserviços).
- Service meshes (malha de serviços): é a camada de infraestrutura que gerencia a comunicação entre serviços de uma aplicação distribuída.
- Plataformas de orquestração: atuam como um maestro. São ferramentas de software projetadas para automatizar e coordenar o gerenciamento de múltiplos sistemas, serviços e tarefas em um ambiente de tecnologia da informação (TI).

Kubernetes (K8s) é uma plataforma open source para orquestração de contêineres que automatiza a implantação, o escalonamento e o gerenciamento de aplicações containerizadas. Ele permite que serviços sejam ajustados em tempo real, com controle de versionamento, roteamento inteligente e políticas de segurança específicas no domínio da aplicação, e oferece alta disponibilidade e autocorreção em clusters de servidores

O uso de infraestrutura como código (IaC), que é uma abordagem que permite o monitoramento contínuo e design centrado no usuário, reforça a capacidade de adaptação dos serviços, garantindo que o ecossistema digital permaneça responsivo, seguro e eficiente frente às mudanças tecnológicas e de mercado. Em

vez de procedimentos manuais, a IaC permite provisionar, configurar e gerenciar infraestrutura de TI usando arquivos de código. Em vez de criar servidores, redes, bancos de dados e serviços clicando em telas de painel, você descreve tudo em código, e a infraestrutura é criada automaticamente a partir do código.

6.5 Métodos, ferramentas e técnicas (MF&T): desenvolvimento ágil em ecossistemas digitais

Quadro 17 – Metodologias ágeis aplicadas em ecossistemas digitais e microsserviços

Método	Aplicação no ecossistema digital
Scrum	Organização de equipes em sprints curtos para entregar incrementos de valor contínuos. Ideal para coordenação entre squads (em português, esquadrão), modelo organizacional usado em metodologias ágeis que define equipes autônomas para o desenvolvimento de diferentes serviços
Kanban	Visualização de quadro ou rede de cartões do fluxo de trabalho e gestão de tarefas em tempo real. Útil para times que lidam com demandas contínuas e manutenção de serviços
SAFe®	Framework de conhecimento e práticas para escalabilidade e implementação de conhecimentos e práticas do ágil em organizações com múltiplos times e produtos interdependentes
DevOps	É o conjunto de práticas culturais e técnicas para integrar desenvolvimento e operações que garantem entrega contínua, automação de infraestrutura e monitoramento constante

Ferramentas populares

Quadro 18 – Metodologias ágeis aplicadas em ecossistemas digitais e microsserviços

Categoria	Ferramenta	Finalidade
Gerenciamento ágil	Trello, Jira, Azure DevOps, ClickUp	Planejamento de sprints, backlog, tarefas e acompanhamento de progresso
Integração contínua/entrega contínua (CI/CD)	GitHub Actions, GitLab CI, Jenkins, CircleCI	Automatização de builds (conversão de código-fonte em software pronto para uso), testes e deploys (implantação) em ambientes distribuídos
Containers e orquestração	Docker, Kubernetes	Empacotamento e escalonamento de microsserviços em ambientes de nuvem
Monitoramento e observabilidade	Prometheus, Grafana, ELK Stack, New Relic	Acompanhamento de desempenho, logs e métricas de serviços distribuídos
APIs e integração	Postman, Swagger/OpenAPI, Kong, Apigee	Design, documentação, testes e gerenciamento de APIs em ecossistemas abertos

Técnicas práticas

As técnicas representam o conjunto de métodos operacionais utilizados no dia a dia do desenvolvimento, operação e evolução de produtos digitais, servindo como ponte entre princípios teóricos e a execução efetiva.

- Desenvolvimento orientado a testes (TDD): garante qualidade desde o início, com testes automatizados para cada funcionalidade.
- Integração contínua (CI): código integrado ao repositório, com testes automáticos para detectar falhas rapidamente.
- Entrega contínua (CD): automatiza a entrega de software em produção com segurança e frequência.
- Infraestrutura como código (IaC): provisionamento automatizado de ambientes operacionais.
- Design de APIs First: planejamento e documentação de APIs antes da implementação, promovendo alinhamento entre equipes.
- Arquitetura de microsserviços: divisão do sistema em serviços independentes, facilitando a escalabilidade e a entrega incremental.
- Pair programming (programação em pares) e code review (revisão de código): pair programming envolve dois desenvolvedores trabalhando juntos em um único código, e code review consiste na inspeção sistemática do código por outros desenvolvedores para identificação de defeitos.
- MVP (produto mínimo viável): lançamento rápido de versões básicas para validação com usuários reais.

Unidade IV

7 VERIFICAÇÃO & VALIDAÇÃO (V&V)

Objetivo da validação é demonstrar que um produto ou componente realmente atende ao propósito para o qual foi criado quando colocado em seu ambiente real de uso. O principal mecanismo tradicional de validação de requisitos são as revisões formais apoiadas por checklists. Entretanto, em contextos ágeis, a validação é ampliada por práticas como demonstrações de sprint (sprint reviews), prototipação rápida, testes de aceitação orientados pelo cliente (acceptance test-driven development – ATDD) e definição de pronto (definition of done – DoD).

A validação demonstra o quanto o produto, na sua forma atual, opera corretamente para o usuário. Para isso, utiliza atividades de verificação como testes, inspeções e simulações.

A **verificação assegura que o produto foi construído corretamente com base nos requisitos do software, especificações e padrões técnicos**, por meio de análises, inspeções, testes automatizados e diagnósticos. Já a **validação confirma**

que o produto entregue é realmente o que o usuário precisa e que funciona adequadamente em seu ambiente operacional.

7.1 Verificação do código: depuração e diagnóstico de falhas

A verificação do código inclui o processo conhecido como depuração. No sentido clássico, depurar significa limpar, remover ou corrigir partes indesejáveis. Durante a construção do software, erros são inevitáveis, seja por travamentos, comportamentos inesperados ou falhas de lógica. Assim, a depuração consiste no rastreamento do código para identificar, corrigir e reduzir essas falhas.

Em termos técnicos, depurar falhas no software (debug) envolve acompanhar a execução do programa em tempo real, prática associada ao termo trace (rastrear).

Depuração de falhas no software (debug)

A depuração de falhas (debugging) é a atividade responsável por rastrear e corrigir esses defeitos, garantindo que o sistema funcione conforme o esperado.

Práticas de CI, testes automatizados, TDD e programação em par (pair programming) reduzem significativamente o surgimento de defeitos e facilitam sua identificação rápida.

Depuração de falhas ilustrada na figura pode ser compreendida em quatro tarefas principais:

- Reprodução (reproduce): consiste em rastrear o erro com o objetivo de identificar a causa do defeito.
- Diagnóstico (diagnose): avalia-se o ponto exato do erro, sua causa, gravidade, prioridade e riscos associados, bem como o impacto da falha ou mudanças que afetam o comportamento do software e em componentes que dependem ou interagem com ele.
- Correção (fix): envolve a implementação da solução e a realização de testes para verificar se o problema foi resolvido.
- Reflexão (reflect): após a correção, medidas são aplicadas para evitar a recorrência do problema. Isso inclui verificar se a mudança não afeta outras partes do sistema, revisar entradas e saídas, otimizar o código e assegurar boas práticas como encapsulamento.

7.2 Testes de software: fundamentos, técnicas e práticas

A lógica adotada segue uma hierarquia natural no ciclo de desenvolvimento. Na sua ordem, o início se baseia em fundamentos conceituais sobre como projetar testes; em uma próxima fase, são apresentados os níveis de testes aplicados ao longo do ciclo de vida do software; e, em seguida, exploram-se práticas orientadas a testes, testes com usuários finais e métodos de garantia de qualidade. Em uma ordem lógica de testes no ciclo de desenvolvimento, é recomendado que:

1) Inicialmente, sejam abordados os testes caixa-preta e caixa-branca, que formam a base metodológica para a elaboração de casos de teste, introduzindo paradigmas que orientam todas as técnicas de testes. 2) Na sequência e em uma ordem natural de testes, são apresentados os testes unitário, de integração, aceitação e regressão, da menor unidade até o produto final, que estruturam a cadeia de verificação do desenvolvimento de software, constituindo a espinha dorsal (backbone) de testes. 3) Na prática do desenvolvimento, organiza-se o ciclo de implementação e testes com o método ágil TDD, redefinindo o papel dos testes unitários ao integrá-los ao ciclo de construção do código. 4) Finalmente, são realizados os testes alfa e beta, para validar soluções em ambiente real e com usuários finais. 5) E na garantia da qualidade, usa-se uma abordagem cleanroom, como um método que prioriza prevenção de defeitos e testes estatísticos de confiabilidade. É um método formal que combina inspeções, desenvolvimento incremental e testes estatísticos.

7.3 Testes caixa-preta e caixa-branca

A primeira perspectiva se baseia no exame das funções especificadas para o produto. Na visão externa, avaliam-se as entradas, saídas e comportamentos observáveis do sistema, com o objetivo de confirmar que cada funcionalidade opera conforme o estabelecido e, simultaneamente, identifica discrepâncias funcionais. Essa abordagem é denominada teste caixa-preta, por prescindir do conhecimento da estrutura interna do software.

A segunda perspectiva fundamenta-se na análise do funcionamento interno do sistema. Na visão interna, investigam-se a arquitetura, a lógica de controle, os fluxos de execução, os módulos e os demais componentes estruturais, de modo a assegurar que todos os elementos internos foram exercitados adequadamente e que suas interações obedecem às especificações técnicas. Essa estratégia caracteriza o teste caixa-branca e se apoia em informações derivadas do código-fonte, assim como em métricas estruturais de software, tais como cobertura de instruções, de decisões e de caminhos.

As duas abordagens são complementares, uma vez que a caixa-preta enfatiza a conformidade funcional e comportamental, a caixa-branca evidencia a qualidade estrutural, o desempenho interno e aspectos de segurança associados à implementação.

Teste caixa-preta, também denominado teste comportamental, é uma técnica de avaliação na qual o software é analisado exclusivamente a partir de suas interfaces externas, sob a ótica do usuário, sem que o testador tenha acesso ou necessidade de compreender sua estrutura interna, lógica de implementação ou arquitetura. O foco dessa abordagem é verificar se o sistema atende aos RF e RNF especificados.

7.4 Testes unitário, de integração, de aceitação e de regressão

O atributo da qualidade testabilidade refere-se à facilidade com que um software pode ser testado, como a clareza de seu design, da modularidade do código, da existência de interfaces bem definidas ou do suporte à automação.

7.4.1 Teste unitário (unit test)

O objetivo do teste unitário é testar componentes pequenos e isolados do software, como funções, métodos ou classes, de forma a assegurar que cada unidade se comporte conforme o esperado.

As ferramentas mais apropriadas a serem utilizadas são as que suportam a criação e execução de testes unitários e o cálculo de métricas de cobertura de código:

- Frameworks de teste unitário: são ferramentas que permitem que o desenvolvedor escreva e execute o código de teste antes do código de produção, conforme o TDD.
- Ferramentas de cobertura de código (code coverage): o método caixa-branca (ou white-box testing) tem o foco na estrutura interna do código. O desenvolvedor precisa saber se o código de teste está realmente executando (e, portanto, verificando) cada linha e cada regra de cálculo. As ferramentas mais apropriadas a serem utilizadas são as que suportam a criação e execução de testes unitários e o cálculo de métricas de cobertura de código.

7.4.2 Teste de integração (integration test)

Após os testes unitários, as unidades são integradas para formar novos componentes ou módulos. Na abordagem ágil, os testes de integração ocorrem de forma incremental, acompanhando as entregas de cada sprint. Seu objetivo é verificar a comunicação e a interação entre módulos, APIs, serviços e componentes do sistema, garantindo que o conjunto funcione de maneira coesa.

7.4.3 Teste de aceitação (acceptance test)

O teste de aceitação, ou teste de validação, ocorre quando o incremento já passou pelos testes unitários e de integração. Seu objetivo é confirmar que o software atende às necessidades do cliente e entrega o valor descrito nos RF e RNF. Estando na sprint review ou em ciclos de validação contínua, o cliente ou product owner realiza o aceite do incremento entregue.

Quadro 21 – Casos de testes caixa-branca que associam o caso de teste com os requisitos do sistema

Evento/momento	Prática de modelagem ágil aplicada
Planejamento da sprint	Modelagem colaborativa/esboços: aqui o objetivo, bem específico, é transformar o requisito em um modelo de comportamento testável. Use um esboço rápido para garantir que todos entendam as interfaces e dependências das histórias de usuário mais complexas
Desenvolvimento (coding)	Modelagem JIT/evolução contínua: o teste de aceitação é aplicado de forma prática através da sua automação e da adoção do TDD, garantindo que o código construído atenda exatamente ao modelo de comportamento definido. Antes dessa atividade, desenhe rapidamente um diagrama de classe no seu IDE ou um diagrama de componentes em até mesmo um papel antes de uma refatoração crucial
Integração/testes (CI)	Modelos de trabalho: o estágio de integração e testes contínuos é onde o teste de aceitação tem sua maior importância, por automatizar o ciclo de desenvolvimento ágil, transformando-se em um modelo de validação da arquitetura do sistema. A cobertura de código se torna os modelos de comportamento e qualidade do sistema
Revisão da sprint (review)	Modelar com propósito: serve de critério final de verificação para que o product owner e stakeholders validem se o incremento de software atende às expectativas e aos requisitos de negócio. Use um diagrama simples (como um diagrama de implantação) para explicar como a nova funcionalidade foi integrada ao ambiente

7.4.4 Teste de regressão (regression test)

O teste de regressão garante que mudanças no código, como correções, melhorias ou novas funcionalidades, não introduzam erros em partes já estáveis do sistema.

A técnica de depuração reversa inverte o fluxo tradicional de análise. Ela ocorre no sentido inverso de rastreamento, ou seja, parte da informação, que é o resultado final, e caminha para os dados que a geraram, resultantes de algoritmos e interfaces usadas na construção da informação. Essa é uma abordagem top-down que verifica a integração dos componentes de um sistema, começando pelos níveis superiores, principalmente as que fazem interface com o operador do software, e caminha para os níveis inferiores, incluindo o projeto e testes do código.

7.5 Design e comportamento: FDD, DDD e abordagens orientadas a testes

As práticas de design e comportamento representam uma sinergia essencial para mitigar riscos e otimizar a construção de software. A análise e intersecção de metodologias como o Feature-Driven Development (FDD), ou Desenvolvimento Dirigido por Funcionalidade, que estrutura o processo em torno de entregas de valor, e o Domain-Driven Design (DDD), ou Design Orientado a Domínio, que fornece os fundamentos arquiteturais para modelar domínios complexos, permitindo a criação de sistema de software robusto e alinhado à necessidade do negócio, essas

abordagens criam uma estrutura coesa para o projeto, garantindo que o escopo (FDD) e o modelo interno (DDD) estejam alinhados às necessidades do negócio. A complementação dessa estrutura ocorre por meio das abordagens orientadas a testes, que transformam requisitos em especificações executáveis, garantindo a qualidade em diferentes níveis. Enquanto o Acceptance Test-Driven Development (ATDD), ou Desenvolvimento Guiado por Testes de Aceitação, e o Behavior-Driven Development (BDD), ou Desenvolvimento Guiado Pelo Comportamento, focam na colaboração para definir e automatizar o comportamento esperado do sistema a partir da perspectiva do usuário. O Test-Driven Development (TDD), ou Desenvolvimento Guiado por Testes, impulsiona o design do código-fonte, assegurando a confiabilidade e a manutenibilidade das unidades de software.

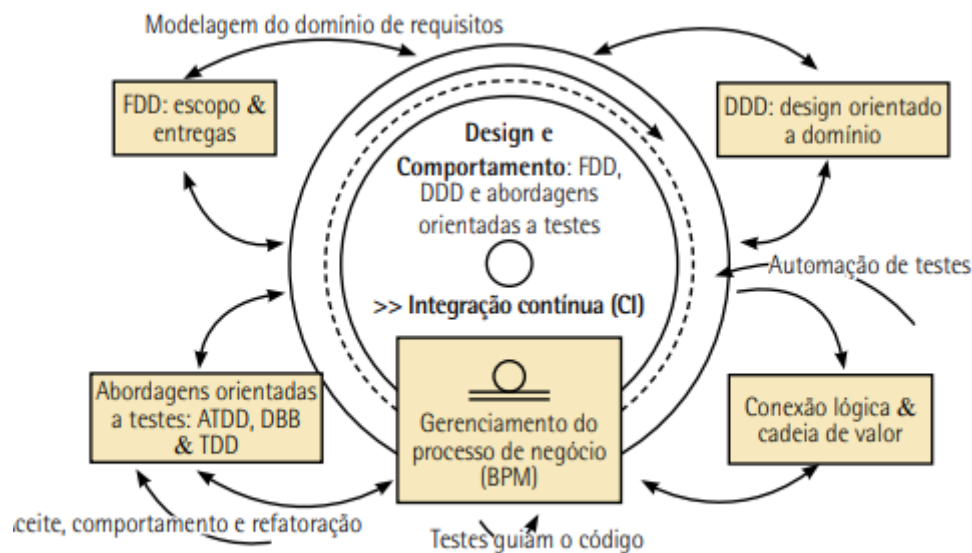


Figura 42 – Design e comportamento: FDD, DDD e abordagens orientadas a testes

- Fundamentos e metodologias ágeis de alto nível: são as estruturas de desenvolvimento que definem o fluxo de trabalho e o ciclo de vida da entrega. O FDD é uma metodologia ágil que organiza o desenvolvimento e o planejamento em torno das funcionalidades (features) do negócio. O FDD utiliza práticas de teste e design com base no TDD/BDD
- Foco na arquitetura e no domínio: o foco está na estrutura do software para lidar com a complexidade do negócio (domínios). O DDD também se beneficia do TDD e BDD para garantir que o modelo do domínio de negócio (entidades, interfaces e controle)

- Práticas orientadas a testes e requisitos (test-driven): detalha as práticas que garantem que o time construa o que é certo de maneira correta.
 - Base da qualidade do código: o TDD é a prática de escrever testes unitários antes do código de produção para impulsionar o design e garantir a qualidade interna do código.
 - Conexão com o negócio: o ATDD é uma filosofia com foco na colaboração para definir os testes de aceitação, que são os critérios de “pronto” do cliente, antes da codificação, de forma a garantir que o time construa a funcionalidade correta.
 - Ferramenta/prática: o BDD é a técnica que implementa o ATDD, usando a sintaxe Gherkin (Dado/Quando/Então) para expressar testes de aceitação em uma linguagem que todos envolvidos no projeto (técnicos e não técnicos) possam entender.
- Medição e execução de processos: é a gestão do trabalho e acompanhamento do desempenho da equipe.
 - Business Process Management (BPM), ou Gerenciamento de Processos de Negócio: é uma disciplina que introduz a modelagem do processo de negócio como base para gerenciar.
- Práticas de entrega contínua: é a prática automatizada do fluxo de trabalho. Aborda:
 - CI/commit no pipeline (envio de mudanças ao repositório): o CI é a prática de integrar o código frequentemente.

7.6 Testes alfa e beta

Quando o software é desenvolvido para apenas um cliente, o teste de aceitação conduzido pelo próprio usuário costuma ser suficiente. Porém, em produtos voltados para múltiplos clientes ou mercados distintos, o teste de aceitação isolado não cobre as variáveis de contexto, diferenças culturais, particularidades regionais ou variações de infraestrutura tecnológica. Dessa forma, a estratégia mais eficaz é incorporar testes alfa e beta.

Teste alfa ocorre em um ambiente controlado, geralmente dentro da própria empresa desenvolvedora. A equipe de desenvolvimento, a Quality Assurance (QA), ou Garantia da Qualidade, e usuários internos (cliente e desenvolvedores) simulam diferentes cenários de uso desde as primeiras etapas do produto.

Teste beta é a validação em ambiente real com feedbacks de usuários finais. Após os ajustes resultantes do teste alfa, o software é liberado para o teste beta, que é realizado no ambiente real do usuário, ou seja, em ambiente descontrolado e sem acesso direto da equipe de desenvolvimento.

Versões beta de software

Os riscos potenciais incluem: • instabilidade operacional: frequentes falhas de sistema, comportamentos inesperados e queda de desempenho; • incompatibilidade no ambiente operacional: problemas de integração e funcionalidade adversa com configurações de hardware e versões do sistema operacionais não validadas; • vulnerabilidades de segurança: presença de bugs de segurança não corrigidos que podem expor o sistema a ameaças cibernéticas; • limitações de suporte: ausência ou redução de suporte técnico, exigindo do usuário a capacidade de diagnosticar e resolver problemas por conta própria; • consumo excessivo de recursos: subutilização de memória, processamento e armazenamento, impactando a eficiência global do sistema computacional.

8 GESTÃO DE MUDANÇAS E EVOLUÇÃO DO SOFTWARE

A gestão de mudanças e a evolução do software constituem elementos estruturais no ciclo de vida de sistemas de software, em contextos organizacionais dinâmicos e altamente dependentes de tecnologias digitais.

8.1 Fundamentos da gestão de mudanças

Entre as particularidades que distinguem a engenharia de software das demais áreas da engenharia, destaca-se o caráter contínuo de manutenção e evolução dos sistemas. Diferentemente de produtos físicos, cuja expectativa é chegar ao usuário totalmente pronto e funcional, o software passa por ajustes desde sua implantação, seja para resolver problemas de configuração, adequações ao ambiente operacional, incompatibilidades de interface ou atender novos requisitos. Isso ocorre porque o software é um artefato vivo: usuários descobrem novas necessidades, identificam oportunidades de melhoria e demandam maior usabilidade, funcionalidade e integração, que são fatores que impulsionam mudanças contínuas no software.

Manutenção de software => constitui o conjunto estruturado de modificações realizadas após sua liberação, englobando correções, melhorias, adaptações e evolução funcional.

A manutenção corretiva, preventiva ou adaptativa permite ajustes conforme as mudanças tecnológicas, correções e demandas de negócios e dos usuários.

Manutenção corretiva

Tem como objetivo restaurar o software ao seu comportamento funcional esperado, garantindo que atenda aos requisitos e expectativas dos usuários. Grande parte dessa atividade corresponde ao processo de depuração de falhas (debugging, comentado no tópico 7.1 Verificação do código: depuração e diagnóstico de falhas).

Integra o uso de ferramentas de debug com técnicas estruturadas de análise de causa raiz (do inglês root cause analysis – RCA). A técnica sugerida é o diagrama de causa e efeito (diagrama de ishikawa ou diagrama de espinha de peixe).

Diagrama de causa/efeito tem por objetivo visualizar, categorizar e mapear as possíveis causas que contribuem para um efeito específico (ou problema).

Manutenção preventiva

São ações proativas que visam evitar problemas futuros e aumentar a robustez do software. Inclui atividades como: • análise de desempenho e melhorias; • revisões de código (code review), especialmente orientadas por padrões de qualidade; • atualizações regulares de segurança (patching); • refatoração contínua para melhorar legibilidade, modularidade e manutenibilidade do código; • verificação antecipada de riscos técnicos.

8.2 Evolução, integração e tendências do software

8.2.1 Abordagens ágeis para manutenção e evolução

Frameworks como Scrum e Kanban promovem visibilidade do trabalho, priorização dinâmica de demandas e ciclos curtos de entrega, permitindo que a equipe responda rapidamente a defeitos, melhorias e novas necessidades de negócio.

A CI/CD, o TDD e a automação de testes garantem que cada alteração seja validada rapidamente, minimizando retrabalho e encurtando o tempo entre o surgimento de

uma demanda e sua implementação. A cultura DevOpsSec complementa esse cenário ao integrar desenvolvimento, operações e segurança, permitindo liberação de releases frequentes.

8.2.2 Evolução do software

A gestão do backlog e os mecanismos de priorização, juntamente com a definição clara dos papéis organizacionais, formam a base essencial para essa estrutura de trabalho.

O emprego de métricas de fluxo, como lead time e cycle time, constitui uma referência para quantificar a eficiência do processo, identificar gargalos e orientar decisões de capacidade e priorização. Essas métricas permitem que organizações operem com maior previsibilidade, ajustem rapidamente seus processos e sustentem um ritmo de entrega compatível com ambientes competitivos e de rápida mudança.

A gestão da dívida técnica e o enquadramento ágil da manutenção contínua despontam como elementos críticos para o futuro da evolução do software. A dívida técnica é entendida como o acúmulo de decisões de engenharia que prejudicam a manutenibilidade, escalabilidade ou qualidade, passando a ser tratada como um ativo monitorável, devendo ser visível no backlog e priorizada com base em seu impacto estrutural.

8.3 Estratégias estruturais: top-down e bottom-up

- Interface com o usuário (front-end/apresentação): foco na experiência do usuário (UX), usabilidade, acessibilidade e validação de requisitos visuais e funcionais a partir do ponto de vista do cliente.
- Interface com o ambiente operacional (backend/infraestrutura): foco na interação do software com o sistema operacional, banco de dados, APIs de terceiros, desempenho, segurança e alocação de recursos.

Essas duas perspectivas informam o que deve ser testado, levando naturalmente às estratégias de integração top-down e bottom-up.

8.3.1 Top-down: integração orientada ao fluxo de valor

A abordagem top-down de testes de integração (como ilustrado na figura 47) verifica a integração dos componentes, começando pelos níveis superiores até os módulos subordinados de nível inferior.

- Ponto de partida: inicia-se pelos requisitos dos módulos de alto nível, histórias de usuários e uso da interface, ou pelo módulo principal de controle, que lida com as regras de negócio de nível superior (por exemplo: característica de um checkout).
- Stubs (simuladores): são a validação da funcionalidade de ponta a ponta (end-to-end). Servem para que o módulo superior possa ser testado antes da conclusão dos módulos inferiores, ou seja, o foco é garantir que o fluxo de valor definido na user story (por exemplo: serviço de como o usuário pode efetuar um pagamento) funcione, mesmo que os componentes inferiores sejam inicialmente stubs. Um stub é um substituto de um código que simula a saída de um módulo dependente.
- Fluxo: o foco é testar o fluxo de controle do sistema, caminhando da decisão (o que o sistema deve fazer) para o detalhe (como o código executa essa tarefa).

Embora o ágil priorize os testes unitários (bottom-up) para a qualidade do código, o top-down se manifesta nos testes de integração de serviços e testes de ponta a ponta (end-to-end).

A técnica top-down verifica a integração dos componentes de um sistema, começando pelos níveis superiores (como requisitos, funções e interface do usuário) e seguindo, progressivamente, para os níveis inferiores (incluindo testes do código, respostas de interfaces e rede).

8.3.2 Bottom-up: integração orientada ao componente

A abordagem bottom-up (ilustrada na figura 48) é mais comum no ágil moderno e verifica os componentes, começando pelos níveis mais baixos e granulares (código e módulos) antes de movê-los para cima.

- Ponto de partida: inicia-se pelos módulos atômicos ou de menor nível, que realizam funções específicas e detalhadas (exemplo: módulos de acesso a dados ou lógica de cálculo).
- Fluxo de teste: os módulos de nível inferior são combinados em clusters (conjuntos) e testados antes de serem integrados aos módulos de controle de nível superior.
- Uso de drivers (controladores): como os módulos de controle superiores ainda não foram

integrados, o teste bottom-up utiliza drivers simuladores. Um driver simulador é um programa dummy que chama e controla o módulo que está sendo testado, fornecendo-lhe dados de teste. A função desse driver é, justamente, simular o ambiente de chamada do módulo superior para garantir que o módulo de baixo nível funcione corretamente sem interrupção. • Desdobramento: os clusters de módulos testados (agora funcionando) substituem os drivers, e o processo continua subindo na hierarquia até que o módulo principal seja finalmente testado.

É uma técnica que inicia os testes de integração pelos módulos de nível inferior, tipicamente em nível de código. Começando pela validação das funcionalidades mais granulares, como a interface com hardware, drivers e instalação, e seguindo, progressivamente, para os níveis superiores (níveis operacionais do sistema).

8.4 Gestão de configuração do software

A gestão de configuração (CM) trata das políticas, processos e ferramentas para gerenciar sistemas de software em constante mudança. É necessário gerenciar sistemas em evolução porque é fácil perder o controle sobre quais alterações e versões de componentes foram incorporadas em cada versão do sistema. A gestão de configuração do software (software configuration management – SCM) controla diversas versões que podem estar em desenvolvimento e uso ao mesmo tempo.

Quadro 23 – Principais atividades do gerenciamento de configuração do software

Atividades	Descrição
Controle de versão	Envolve rastrear as múltiplas versões dos componentes do sistema e garantir que alterações feitas por diferentes desenvolvedores não interfiram umas nas outras
Gestão de mudanças	Consiste em registrar solicitações de mudança referentes ao software entregue por clientes e desenvolvedores, analisar custos e impactos dessas mudanças e decidir se e quando devem ser implementadas
Construção de sistemas	É o processo de reunir componentes de programa, dados e bibliotecas, compilando e vinculando esses elementos para criar um sistema executável
Gestão de releases	Envolve preparar o software para lançamento externo e manter o controle das versões do sistema que foram disponibilizadas para uso pelos clientes

8.4.1 Versionamento do software

O controle de versões alinhado ao controle de releases formam os processos responsáveis por registrar, organizar e acompanhar a evolução dos artefatos do software ao longo do tempo, possibilitando a rastreabilidade das modificações realizadas. Esse processo, conhecido como versionamento, permite identificar claramente cada estado evolutivo do sistema.

Na engenharia de software, a **versão refere-se a uma configuração específica do software que incorpora novas funcionalidades**, correções ou adaptações, e que pode estar destinada ao uso interno para desenvolvimento, testes ou integração. Já o **release** é uma **versão formalmente preparada e estabilizada para distribuição** ao cliente ou implantação em ambiente de produção.

Quadro 24 – Técnicas básicas de numeração e identificação da versão aplicadas no gerenciamento de versões

Técnicas básicas	Descrição
Numeração de versões (version numbering/ semantic versioning)	É o esquema de identificação mais comum. Atribui-se um número, explícito e único, de versão ao componente. Em práticas ágeis, costuma seguir o modelo major.minor.patch, facilitando integrações frequentes e rápida compreensão do impacto da alteração Exemplo: Versão 3.21 – 3 representa mudança estrutural significativa (major), 2 indica incremento funcional (minor), e 1 refere-se à correção de defeitos ou pequenos ajustes (patch)
Identificação baseada em atributos (attribute-based identification)	Cada componente recebe um identificador associado a atributos específicos (tipo, funcionalidade, módulo, variante, sprint ou branch). Essa estratégia é útil em ambientes ágeis com múltiplos times ou domínios, permitindo organização modular e rastreabilidade refinada Exemplo: Versão TEC01_1 – TEC – representa o módulo teclado, 01 inserção de um mapa de caracteres (incremento funcional) e _1 indica ajuste ou correção
Identificação orientada a mudanças (change-oriented identification)	Relaciona explicitamente a versão a uma ou mais solicitações de mudança, histórias de usuário ou itens do backlog. Em métodos ágeis, facilita o rastreamento direto entre commits, tarefas e entregáveis Exemplo: CAR_04 – mudança associada à quarta solicitação registrada por "Carlos" ou ao item 04 do backlog do módulo correspondente
Identificação vinculada ao pipeline ágil (commit/build tagging)	Utilizada em DevOps e integração contínua. Cada alteração é identificada por tags, hashes de commit (exemplo: Git) ou identificadores automáticos do pipeline. Garante rastreabilidade completa da origem da mudança e acelera correções Exemplo: build-2025.04.15-#842 ou commit a7f9c2b
Versões orientadas à entrega (release-based identification)	Identificadores próprios para releases estáveis, úteis em modelos ágeis com entregas frequentes. Podem ser alinhados ao planejamento de sprints, releases trimestrais ou marcos do produto Exemplo: Release 2025.Q1, Sprint-12-Release

Um release é uma versão formal do software autorizada para distribuição ao cliente, resultado de um processo controlado que valida estabilidade, segurança e completude funcional. A preparação de um release envolve uma série de elementos técnicos e organizacionais, tais como: instaladores devidamente empacotados, manuais técnicos e de usuário revisados, arquivos de configuração padronizados, além de bibliotecas, conjuntos de dados e scripts de registro necessários para garantir sua correta instalação e operação no ambiente do cliente.

A numeração do release não depende diretamente da sequência de versões internas.

8.5 Métodos, ferramentas e técnicas (MF&T): controle do código-fonte com GitHub

O principal propósito do GitHub é facilitar a colaboração e o controle de versão de projetos de software. Basicamente ele serve para:

- Hospedagem de repositórios: armazenar o código-fonte de projetos em um local central e seguro na nuvem, acessível por qualquer pessoa autorizada.
- Colaboração distribuída: possibilitar que várias pessoas (equipes ou comunidades open source) trabalhem no mesmo código simultaneamente, sem interferir umas nas outras.
- Controle de versão: usar o Git para rastrear todas as alterações feitas no código, permitindo que a equipe volte a qualquer versão anterior, se necessário.
- Gerenciamento de projetos: oferecer ferramentas para planejar, rastrear e documentar o desenvolvimento do software.

Quadro 25 – Funcionalidades do GitHub

Funcionalidade	Descrição
Repositórios (repos)	Onde todo o código, histórico (commits) e arquivos do projeto são armazenados. Podem ser públicos (visíveis para todos) ou privados. Cada commit representa uma mudança no código
Branches (ramificações)	Permite que desenvolvedores criem cópias isoladas do código principal (main) para trabalhar em novas funcionalidades ou correções sem afetar a versão estável. O trabalho com branches (ramificações) permite manter múltiplas linhas de evolução do software, exatamente como em um grafo de versões. Os exemplos mais comuns são: main (versão estável do sistema), develop (linha de integração, modelo GitFlow; feature/ (novas funcionalidades), fix/ (correção de erros) e release/ (preparação de releases)
Pull requests (PRs)	É um mecanismo central de colaboração para integrar código entre branches. É o pedido formal para integrar as mudanças de um branch de volta ao branch principal. Permite revisão de código antes da fusão
Revisão de código	Permite que outros membros da equipe revisem, comentem e sugiram alterações no código dentro de um pull request antes que ele seja aceito

Quadro 26

git init	; Inicia um novo repositório Git com loco no diretório
git add .	; Adiciona todos os arquivos novos, modificados e excluídos do diretório de trabalho para a área de staging
git commit -m "Versões e Releases"	; Grava as alterações preparadas na área de staging no histórico do repositório local
git push -u origin main	; Envia os commits locais para o repositório remoto